

Degree Project in Computer Science and Engineering

Second cycle, 30 credits

Balancing Adaptation and Forgetting in Tabular Foundation Models under Concept Drift

A Study of Context Memory

WENBO XIA

Balancing Adaptation and Forgetting in Tabular Foundation Models under Concept Drift

A Study of Context Memory

WENBO XIA

Date: October 26, 2026

Supervisor: Tianyu Wang
Examiner: Niharika Gauraha
School of Electrical Engineering and Computer Science
Swedish title: Balans mellan anpassning och glömska i tabulära grundmodeller under begreppsdrift
Swedish subtitle: En studie av kontextminne

Balancing Adaptation and Forgetting in Tabular Foundation Models under Concept Drift

A Study of Context Memory

WENBO XIA

Tabular foundation models such as TabPFN learn from labelled examples in their context, without weight updates, and are now applied to streams whose patterns change over time (concept drift). The context is thus the model's only memory of the stream, and its size is bounded. Forgetting matters because earlier situations can return, yet accuracy on incoming data cannot reveal it. In a study on four public streams, pre-registered with documented amendments, we measure adaptation (accuracy on incoming rows) and retention (accuracy on held-out data of earlier periods, or regimes). On two streams, an existing class-balanced memory for this setting in effect keeps only the most recent rows. We propose a regime-balanced memory that shares the long-term part among earlier regimes; given their boundaries, it raised retention over the class-balanced memory by 1.4 to 18.0 percentage points on all four streams, in every run, at a stream-dependent cost in adaptation of up to 2.1 points. Evaluations should therefore report retention, and memories should keep rows of every past regime that may return.

SAMMANFATTNING

Tabulära grundmodeller (foundation models) som TabPFN lär sig av märkta exempel i sin kontext, utan att vikterna uppdateras, och används nu på dataströmmar vars mönster förändras över tid (begreppsdrift). Kontexten är alltså modellens enda minne av strömmen, och dess storlek är begränsad. Glömskan spelar roll eftersom tidigare situationer kan återkomma, men träffsäkerheten på inkommande data kan inte visa den. I en studie på fyra öppet tillgängliga dataströmmar, förregistrerad med dokumenterade ändringar, mäter vi anpassning (träffsäkerheten på inkommande rader) och bibehållen träffsäkerhet (retention: träffsäkerheten på undanhållna testdata från tidigare perioder, så kallade regimer). På två strömmar behåller ett befintligt klassbalanserat minne för denna tillämpning i praktiken bara de senaste raderna. Vi föreslår ett regimbalanserat minne som fördelar långtidsdelen mellan de tidigare regimerna; när deras gränser är kända ökade det den bibehållna träffsäkerheten jämfört med det klassbalanserade minnet med 1,4 till 18,0 procentenheter på alla fyra strömmar, i varje körning, till en kostnad i anpassning som beror på strömmen och är högst 2,1 procentenheter. Utvärderingar bör därför redovisa den bibehållna träffsäkerheten, och minnen bör behålla rader från varje tidigare regim som kan återkomma.

Keywords: Concept drift, Tabular foundation models, In-context learning, Continual learning, Catastrophic forgetting, Data streams

Nyckelord: Begreppsdrift, Tabulära grundmodeller, Kontextinlärning (in-context learning), Kontinuerlig inlärning, Katastrofal glömska, Dataströmmar

Author's address: Wenbo Xia, wenbox@kth.se, KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, Stockholm, Sweden, SE 100 44.

Document date: October 26, 2026

Swedish title: Balans mellan anpassning och glömska i tabulära grundmodeller under begreppsdrift

Swedish subtitle: En studie av kontextminne

© 2026 Copyright held by the author.

Contents

Abstract 1
Sammanfattning 1
1 Introduction 3
1.1 Background: Learning from Data That Change 3
1.2 Problem: What Should a Bounded Context Keep? 3
1.3 Research Questions and Contributions 5
1.4 Scope and Structure 5
2 Background and Related Work 6
2.1 Concept Drift and Learning from Data Streams 6
2.2 Continual Learning: Stability, Plasticity and Replay 6
2.3 Tabular Foundation Models and In-Context Learning 7
2.4 Tabular Foundation Models on Data Streams 7
2.5 Positioning of This Thesis 8
3 Method 8
3.1 Research Process and Change of Research Direction 8
3.2 Learning Setting and Evaluation Protocol 9
3.3 Context Memory Policies 10
3.4 Data Streams 10
3.5 Experimental Design and Statistical Analysis 12
4 Results 12
4.1 RQ1: Retention Differs Much More Than Stream Accuracy 12
4.2 RQ2: Class-Balanced Memory Collapses to a Recency Window on Two Streams 14
4.3 RQ3: Regime-Balanced Allocation Reduces Forgetting at a Stream-Dependent Cost 15
4.4 Exploratory Analysis: Shares of the Short- and Long-Term Parts 18
5 Discussion 18
5.1 Why Regime Balancing Reduces Forgetting 18
5.2 When Does It Help, and What Does It Cost? 18
5.3 When Is Retention Useful? 20
5.4 Relation to Existing Work 20
5.5 Limitations 20
6 Conclusions and Future Work 21
Acknowledgments 22
References 23
A Preliminary Study: Three-Level System and Drift Detection 25
B Dataset Details and Reproducibility 26
C Pre-registration and Hypotheses 29
D Supplementary Results 30

1 INTRODUCTION

Tabular foundation models predict new rows from labelled example rows without any training, and they are now being applied to data that change over time. Their weights stay fixed after pre-training, so such a model learns a particular stream only from the rows it is currently shown, and only a limited number fit. Which rows should it keep? This thesis shows that an existing way of choosing them can silently forget the past, that a simple change keeps the past better on all four streams studied, by up to 18 percentage points, and that the difference is visible only when forgetting is measured.

1.1 Background: Learning from Data That Change

Many tables are recorded over time, such as gas-sensor readings or electricity prices. In such data the relation between the inputs and the correct label can change, so that the same input calls for a different answer than before. This is *concept drift* [12, 36]. We call a stretch of a stream in which the relation stays the same a *regime*, and the point where one regime ends a *change point*. A change can be abrupt, gradual or incremental, and earlier regimes can recur [12, 36, 50]; Figure 2 illustrates the four types.

A learner on a stream must adapt quickly to the current regime, yet keep what it learned about earlier ones, which may be needed again. This tension between *plasticity* (taking in the new) and *stability* (keeping the old) is the stability–plasticity dilemma, studied since Grossberg [16] [see also 39].

Tabular foundation models change where this tension lies. A model such as TabPFN is pre-trained once, on a very large number of synthetic prediction tasks, and then used with frozen weights [15, 20, 21, 43]. For a new task it receives labelled rows, its *context*, and predicts new rows in one forward pass. This is *in-context learning*: like a student shown worked examples before an exercise, the model learns the task from the examples, without training. On a stream, the weights hold only the general prior from pre-training, so *the context is the model's only memory of the stream*, and a row pushed out of it is lost for good, as in instance-based stream learners [32]. For large tabular models, "the memory population effectively is the model" [34]: choosing the context rows decides the balance between stability and plasticity [34, 44], and the challenge shifts from updating the model to managing its context [27, 53].

The context cannot grow without limit, because the cost of a prediction grows with its size [20] (on our longest stream, about 14 minutes per run with 200 context rows and half an hour with 1000). A stream is therefore served with a fixed *context budget*, in which recent rows favour adaptation and older rows memory of the past.

1.2 Problem: What Should a Bounded Context Keep?

We describe a model on a stream by two quantities, measured as shown in Section 3.2. *Adaptation* (ADAPT) is its accuracy on the incoming rows, measured *prequentially*: each row is predicted before its label is revealed and only then used for learning [11]. *Retention* (RET) is the accuracy that the current context gives on *holdout sets*, rows of each earlier regime kept out of the stream for this test only. All accuracies are *balanced*, the mean of the per-class accuracies [3], so that guessing the most common class scores poorly; differences are in *percentage points* (pp).

Figure 1 shows the three policies. A *sliding window* keeps the most recent rows. A *dual memory* splits the budget into a *short-term part* of recent rows and a *long-term part* that receives the rows leaving it; when the long-term part is full, a row is removed (*evicted*). The class-balanced dual memory (Lourenço et al. [35]) evicts the oldest row of the class with most rows. When the classes arrive mixed, without long runs of a single class, this evicts almost in order of age, and only the latest regime survives. We call

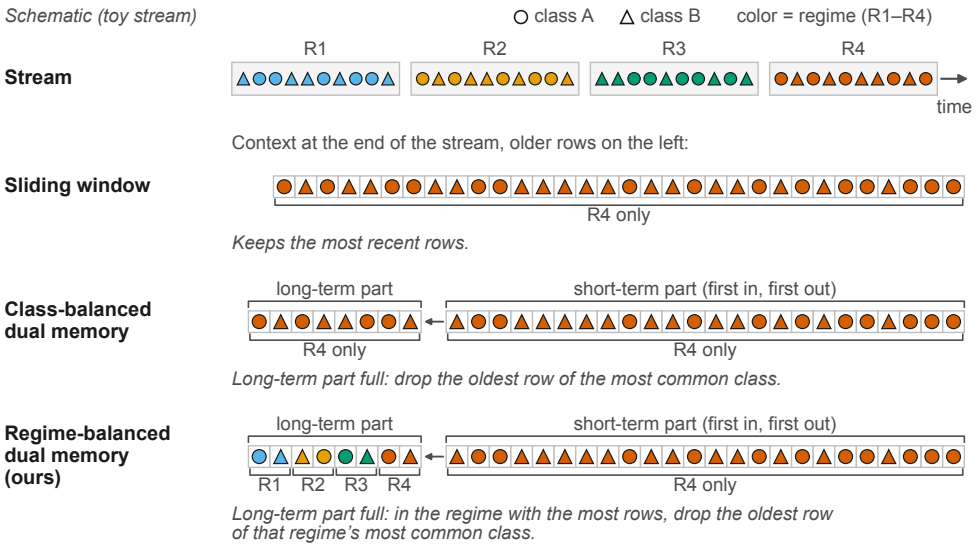


Figure 1: Which rows three context policies keep at the end of a stream (schematic). A toy stream passes through four regimes, R1–R4 (colour), with two equally frequent classes (shape) mixed in every regime; each 32-row memory is shown from older (left) to newer (right) rows, and in the two dual memories the oldest row of the short-term part moves into the long-term part. Under the sliding window and the class-balanced dual memory only rows of the latest regime remain, whereas the regime-balanced dual memory keeps rows of every regime in its long-term part (contents produced by the eviction rules used in the experiments).

this *recency collapse*; in the strict sense, a retention within 0.3 pp of a sliding window of the same size, we observe it on two of our four streams, Gas and Electricity. On Insects (abrupt), where a long run of one class precedes each change point, some old rows survive (Section 4.2). The regime-balanced dual memory (ours) evicts first from the regime with most rows, and so shares the long-term part among all regimes seen so far.

Why remember the past? Earlier situations come back, as seasons do, and must then be predicted before new labels arrive, from the old rows still in the context; RET measures this moment. This matters to those who deploy tabular foundation models on changing data, as in sensor, energy or environmental monitoring, because a model that has silently forgotten an earlier situation can fail exactly when it returns. It also matters to those who evaluate stream learners, since accuracy on the stream scores only the current regime and so hides forgetting unless retention is measured: on Gas, two dual memories that differ only in what their long-term part keeps differ by $+0.09$ pp in ADAPT but by $+18.00$ pp in RET (200 rows). Yet the class-balanced dual memory, proposed for TabPFN on streams [35], has been compared with other policies by prequential measures only [53]. This thesis therefore asks: *given a fixed context budget, which rows should a frozen tabular foundation model keep so that it adapts to the current regime without forgetting earlier ones, and how can such forgetting be measured?*

1.3 Research Questions and Contributions

RQ1 How much does a frozen tabular foundation model forget earlier regimes of a concept-drift stream under different context-memory policies (sliding window, class-balanced dual memory, regime-balanced dual memory), and how does this relate to its accuracy on the incoming stream?

RQ2 Why does an existing class-balanced context memory forget earlier regimes, and on which of our streams does it reduce to keeping only the most recent rows (recency collapse)?

RQ3 Does allocating the long-term context capacity by regime reduce forgetting on streams with different drift characteristics, and at what cost in adaptation?

The thesis makes three contributions, each stated as what we did, what we found and what is new.

(1) *Making forgetting visible.* We measure retention next to adaptation, as streaming continual learning does [6], in a study pre-registered with documented amendments. The policies differ by up to 18.2 pp in RET but by at most 3.5 pp in ADAPT. *New here:* this measurement for a frozen tabular foundation model with a fixed context budget on drift streams. The studies of such models on streams that we found report only prequential or online metrics [7, 27, 35, 53, 54]; the nearest study of forgetting uses image task sequences and a memory that only grows [44].

(2) *Explaining why an existing memory forgets.* The class-balanced dual memory balances classes, not regimes. On Gas and Electricity, where the classes arrive mixed, it keeps almost only the most recent rows: 94.7–97.5 % of its rows are those a sliding window of the same size would hold, and its RET is within 0.3 pp of that window (Table 3). *New here:* this failure, recency collapse, and its cause in the eviction rule.

(3) *A simple remedy, given regime boundaries.* The regime-balanced dual memory gives every past regime a share of the long-term part. It raises RET over the class-balanced dual memory on all four streams and at both budgets, by $+1.42$ to $+18.00$ pp, in all ten runs of each comparison, at a cost in ADAPT that depends on the stream: near zero on Gas and Insects (abrupt), largest on Electricity (-2.10 pp at 200 rows). *New here:* not the allocation principle, which comes from replay memories in continual learning [4, 5, 31, 46], but its use for the context of a frozen model, with drift regimes as the groups.

1.4 Scope and Structure

We study TabPFN because it is accurate on small tables without tuning [21], never trained during use, keeps a memory of rows that can be inspected one by one, and is used, alone or among other backbones, by the related studies on streams [7, 27, 35, 53]. We use one frozen backbone (the TabPFN-2.5 weights), classification only, four streams, contexts of 200 and 1000 rows and batches of 50 rows. The main method is given the regime boundaries; a variant that detects them is run on two streams. Stream learners that are not foundation models are not compared (limitations in Section 5.5).

Section 2 reviews related work (summary in Table 1). Section 3 tells how the research arrived at its questions and gives the protocol, policies, streams and statistics, including data ethics and research integrity. Section 4 answers RQ1–RQ3 and explores the shares of the short- and long-term parts. Section 5 explains the results, relates them to earlier work and collects the limitations, together with the sustainability and ethical implications of keeping or forgetting the past. Section 6 sums up the findings and their implications and outlines future work. The appendices give the preliminary study, dataset details, the pre-registration and supplementary results.

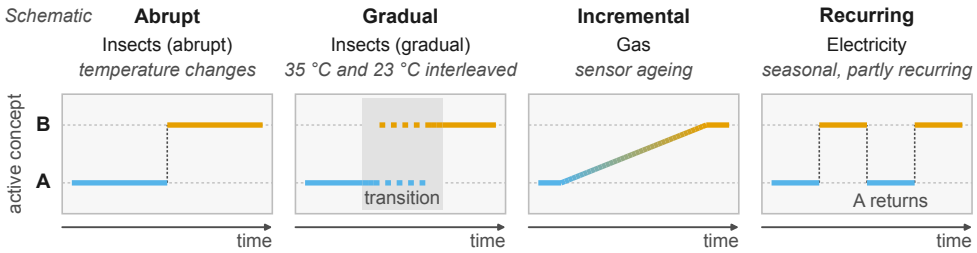


Figure 2: Four types of concept drift (schematic). The vertical axis shows which concept generates the data. Abrupt: the concept changes at one point in time. Gradual: during a transition, instances of the old and the new concept are interleaved, and the new one becomes more frequent. Incremental: the concept moves step by step from A to B. Recurring: an earlier concept returns. The line under each panel title names the stream of this study that has this type of drift (Table 2).

2 BACKGROUND AND RELATED WORK

This section introduces the ideas the thesis builds on and shows what earlier work leaves open. Five lines of work bear on the question; for each we state its usual approach, why it leaves our question open, and what our design takes from it (summary in Table 1). In brief, we found no study that measures what a frozen tabular model with a bounded context forgets on a drifting stream.

2.1 Concept Drift and Learning from Data Streams

In stream learning, each arriving row is predicted before its label is known, and the data drift in the ways of Figure 2. The common approach reacts to drift: detect the change, then forget outdated data or models. ADWIN keeps a window of recent observations whose length adapts to the rate of change [2]; adaptive random forests replace a tree when its detector fires [13]. Windowing assumes that the most recent data are the relevant ones, which fails when concepts recur [12]; recurring concepts are handled by storing whole models and reusing them [10, 14, 24]. Evaluation is prequential (test-then-train), often with fading factors that deliberately forget old errors [11]; the INSECTS benchmark, the source of two of our streams, is evaluated prequentially [50].

Instance-based learners, whose model is the set of stored examples, are the closest analogue to a frozen foundation model: SAM-kNN keeps a short-term memory for the current concept and a long-term memory of older, non-contradicting examples for recurring ones [32, 33], an analogy drawn by Verma et al. [53]. Streaming continual learning joins the rapid adaptation of stream learning with the retention sought by continual learning [6, 17]: Cossu et al. [6] score prequential Cohen's kappa (accuracy corrected for chance) and, after each drift, kappa on held-out sets of all earlier concepts, but for learners that update parameters. In other stream-learning work, accuracy on earlier concepts is not a standard measure [12, 36]. We adopt the two-sided protocol for a model whose memory is a fixed-size context.

2.2 Continual Learning: Stability, Plasticity and Replay

Continual learning studies models that learn a sequence of tasks, framed as the stability–plasticity dilemma [16, 39, 45]. Its failure mode is *catastrophic forgetting*: training on new data overwrites the weights that encoded earlier knowledge [9, 38]. Remedies protect important weights [25] or replay examples from a small memory, and forgetting is measured as the drop in accuracy on earlier tasks (backward transfer) [31]. Replay memories are balanced by class [5, 46, 48], by task [31] or by both [4];

replay designed for drift tends to remove drifted examples rather than protect them [1, 26]; prompt-based methods freeze the backbone but learn small prompts [55, 56].

All of these update parameters, so the question of what the memory alone lets a frozen model retain does not arise. We borrow two allocation principles, partitioning the memory and evicting from the most represented group, and apply them to the context, with drift regimes as the partition. The order of eviction matters: class-balanced reservoir sampling evicts a random row of the largest class and GDumb refuses new rows of full classes, so neither favours recent rows [5, 46]; evicting the *oldest* row of the largest class, as the class-balanced dual memory does, can.

2.3 Tabular Foundation Models and In-Context Learning

A prior-data fitted network is a Transformer trained once on many synthetic datasets so that one forward pass approximates Bayesian prediction for a new dataset [43]. TabPFN applies this to tables [15, 20, 21]; TabICL is a related model for larger tables [47]. Without positional encoding, row order carries no information [43], so what matters is which rows the context holds. Context selection targets large static datasets: retrieving neighbours of each query [51], routing queries to data subsets [57] or tuning a compressed context [8]. Temporal shift has been addressed through the pre-training prior [19], and the TabPFN authors list data drift as future work [21]. What a bounded context should keep on a drifting stream is open; we keep the model fixed and treat its context as a time-ordered memory with a budget chosen for the stream.

For language models the prompt likewise carries task knowledge to a frozen model. Studies ask which demonstrations to retrieve [29, 49], what they convey [40] and how order and position bias the answer [30, 37, 58]; a recent sub-line measures forgetting when a frozen language model learns tasks or classes in sequence through its prompt [23, 41], with a theory for linear attention [28]. There, old classes keep their meaning and nothing is evicted under a budget. The recency bias of language models lies in the *reader*, which weights the end of a prompt [58], whereas recency collapse lies in the *memory policy*.

2.4 Tabular Foundation Models on Data Streams

Lourenço et al. [35] combine a short-term FIFO queue (first in, first out) with a long-term store balanced over classes, meant to keep representative samples of recurring classes. Evaluated prequentially only, the dual memory is on average 0.31 pp more accurate than the short-term memory alone, with no significant difference on five of seven streams [35]. Its authors call the design naive, its stability addressing forgetting of classes only [34]. CURE admits rows by the model's uncertainty and evicts a redundant row of the most represented class [27]; HINT keeps a sliding window on an edge device and sends uncertain rows to TabPFN in the cloud [7]. Wang et al. [54] correct the predictions of a frozen in-context tabular model under temporal drift with a nearest-neighbour residual corrector in its embedding space, fed by past labelled rows; on eight industrial streams they attribute the gains to four mechanisms and characterise theoretically what such correction can and cannot recover. Their adaptation acts on the outputs rather than the context rows and is scored on the current error, not on earlier regimes. Narayanan et al. [44] measure forgetting of a frozen TabPFN on image task sequences with a memory that only grows.

The closest work, Verma et al. [53], compares memory policies across tabular foundation models and finds keeping only the most recent rows as effective as the others, including the class-balanced dual memory; memory size matters far more than the policy (their Tables 2 and 3). On their one stream with recurring concepts, TabPFN scores 0.646 with both a FIFO and the dual memory (their Table 9), which

Table 1: The thesis compared with the five lines of related work.

Line of work	Learning happens in	What is kept	Evaluation	Retention of earlier regimes measured?
Stream learning (Section 2.1)	parameters; stored examples in instance-based learners	recent window, pool of models; short- and long-term example memory (SAM-kNN)	prequential	mostly no; yes in streaming continual learning, for parameter-based learners
Continual learning and replay (Section 2.2)	parameters	replay memory balanced by class, task, or task and class	accuracy on earlier tasks, backward transfer	yes, on task sequences
Tabular foundation models (Section 2.3)	context (frozen weights)	one static table	static benchmarks	no
In-context learning in language models (Section 2.3)	context (frozen weights)	one prompt; sequences of tasks or classes	per-query accuracy; old-task accuracy	partly; task or class sequences without drift
Tabular foundation models on streams (Section 2.4)	context (frozen weights)	bounded rows: FIFO, class-balanced dual memory, CURE, retrieval	prequential or online only	no (one image-task study with a growing memory)
This thesis	context (frozen TabPFN)	fixed budget; long-term part balanced by regime (given boundaries)	ADAPT and RET	yes, on tabular concept-drift streams

fits either retention not helping or the dual memory not retaining; prequential accuracy cannot tell. They analyse memory content only analytically, for a full FIFO memory (their Appendix F.1); what a dual memory keeps is not measured, and no policy allocates the context by regime.

2.5 Positioning of This Thesis

Table 1 reads row by row: stream learning mostly scores the current concept, continual learning protects parameters, and work on tabular foundation models for streams reports only prequential or online metrics.[1] The three gaps lead to three design decisions: we measure adaptation and retention together (Section 3.2), we measure what a class-balanced store actually keeps (RQ2), and we test an allocation by regime, given the regime boundaries (Section 3.3).

3 METHOD

This section describes how the study was done: how the research arrived at its questions, how a model is run and scored on a stream, which memories and streams are compared, and how the results are tested.

3.1 Research Process and Change of Research Direction

The project set out to build a three-level system around a frozen TabPFN: a fast corrector of its outputs, a gated adapter adding a small learned correction per regime, and a drift detector whose alarms trigger actions such as resetting the context. Judged by accuracy on the stream, its gains on synthetic streams did not carry over to real ones: in a GPU experiment on Insects with the ADWIN detector [2] of the river library [42], the detector raised alarms, but no action after an alarm gave a measurable gain, and the system made more errors than the frozen model alone (Appendix A). Correcting a frozen tabular model's outputs from past labelled rows has been analysed in depth by Wang et al. [54], who find that it helps on

[1]The literature search was made in October 2026 in OpenAlex, arXiv, Semantic Scholar, Crossref, DataCite and PMLR; Google Scholar, DBLP, the ACM Digital Library and OpenReview were not searched directly. Every reference was checked in two independent sources.

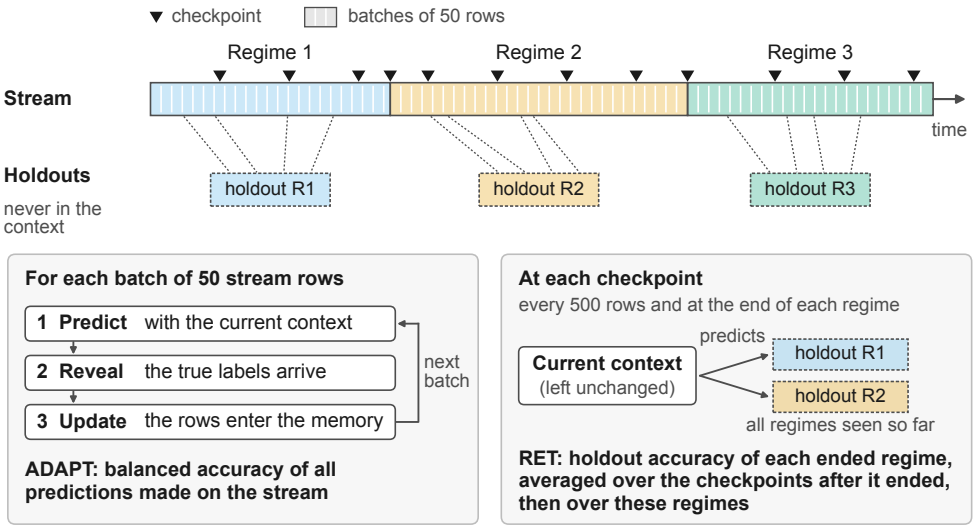


Figure 3: How adaptation (ADAPT) and retention (RET) are measured. The stream is processed in batches of 50 rows: the frozen model predicts each batch from its current context, the true labels are then revealed, and the memory policy updates the context; ADAPT is the balanced accuracy of all these predictions. A holdout set drawn from each regime never enters the context; at every checkpoint (every 500 rows and at the end of each regime) the current context, left unchanged, predicts the holdouts of all regimes seen so far; RET uses those of the regimes that have ended, averaged over checkpoints and then over regimes.

seven of their eight temporal streams but harms on one; our negative results are in line with this stream dependence, although the two correctors differ in design.

A frozen model already adapts through its context, so the question became what the context should keep, and how much the model forgets when it keeps the wrong rows. The new study was *pre-registered*: question, data, methods, metrics and hypotheses were fixed in writing before the main experiments, and every later change was added as a dated amendment with its reason. The method registered at the outset, an archive of rows per finished regime consulted when a query resembles that regime, failed its pilot criterion by losing too much accuracy on the stream, and the pilot showed the class-balanced dual memory retaining little of an early regime; an amendment therefore replaced it by the regime-balanced dual memory before the main runs (Appendix C).

3.2 Learning Setting and Evaluation Protocol

Figure 3 shows the protocol. From row 200 of a time-ordered stream on, rows are processed in batches of 50 that never span a regime boundary: the model predicts a batch from a context of earlier rows only, the labels are revealed, and the memory policy takes in the new rows. Features are standardised with the mean and standard deviation of the first 200 rows. Before the stream starts, a holdout set is drawn from every regime, stratified by class, and removed from the stream. At every *checkpoint* the current memory predicts the holdouts of all regimes seen so far. Run-time checks ensure that checkpoints never change the memory and that the context never holds a future row or exceeds its budget.

ADAPT is the balanced accuracy over all predicted stream rows (random guessing gives $1/K$ for K classes). RET is, for each ended regime, the balanced accuracy on its holdout set averaged over the

checkpoints after its end, then averaged over these regimes. In plain terms, RET asks: if an earlier regime came back now, before any of its new labels arrived, how well would the current context do?

The model is TabPFN with the TabPFN-2.5 weights [15] (tabpfn 6.4.1), frozen, averaging four forward passes on slightly varied inputs. Each setting is run with ten *seeds*, which fix the holdout rows and TabPFN's random state; the stream order never changes. "10/10" means that a difference has the same sign in all ten runs.

3.3 Context Memory Policies

Each policy keeps at most $B = 200$ or $B = 1000$ rows, and all three are compared at the same budget (Figure 1).

Sliding window. The context is the B most recent rows, which amounts to using TabPFN alone.

Class-balanced dual memory. This is the dual memory of Lourenço et al. [35]. The short-term part is a FIFO queue holding 75 % of the budget (150 or 750 rows); the row that leaves it moves into the long-term part (50 or 250 rows). When the long-term part is full, the oldest row of its most common class is evicted (their Algorithm 1). The original study used TabPFN version 1 and memories of 600 to 1000 rows with short-term shares of 0.65 to 0.85.

Regime-balanced dual memory (ours). Only the eviction rule of the long-term part differs. Every row carries the regime in which it arrived, and when the long-term part is full one row is evicted in two levels, regime and then class:

(1) find the regime with most rows in the long-term part (on a tie, the newer regime);
(2) within that regime, find the class with most rows (on a tie, the smaller class index);
(3) evict the oldest row of that class in that regime.

Each regime seen so far thus receives a roughly equal, class-balanced share of the long-term part; without regime boundaries the rule reduces exactly to the class-balanced one (checked by a unit test). It carries two principles of continual learning over to the context: partitioning the memory by task [4, 31], here by regime, and evicting from the most represented group [5, 46].

Regime boundaries. In the main experiments the regime-balanced dual memory receives the documented change points, as the continual-learning strategies of Cossu et al. [6] receive theirs; the other two policies need none. This separates *when* the data changed (detection), a problem of its own on real streams (Section 3.1), from *what* to keep (allocation); in practice boundaries can come from known events, such as a new season, or from a detector. A *boundary-free variant* closes a regime when ADWIN [2], run on the policy's own errors, raises an alarm (river implementation, $\delta = 0.002$, 300 rows of pause after an alarm, regimes of at least 500 rows); it was run on Insects (abrupt) at 1000 rows and on Insects (gradual) at both budgets. On the gradual stream the main runs close R0 at the documented change point; two further variants are in Appendix B.

3.4 Data Streams

Figure 4 and Table 2 describe the four streams, chosen because their drift differs (Figure 2); in Gas the sensors age and earlier conditions do not return, while Electricity's seasons partly recur [18]. As to data ethics, all four are public research benchmarks, recorded from insect sensors, gas sensors and an electricity market; none contains data about persons, and their sources, and licences where stated,

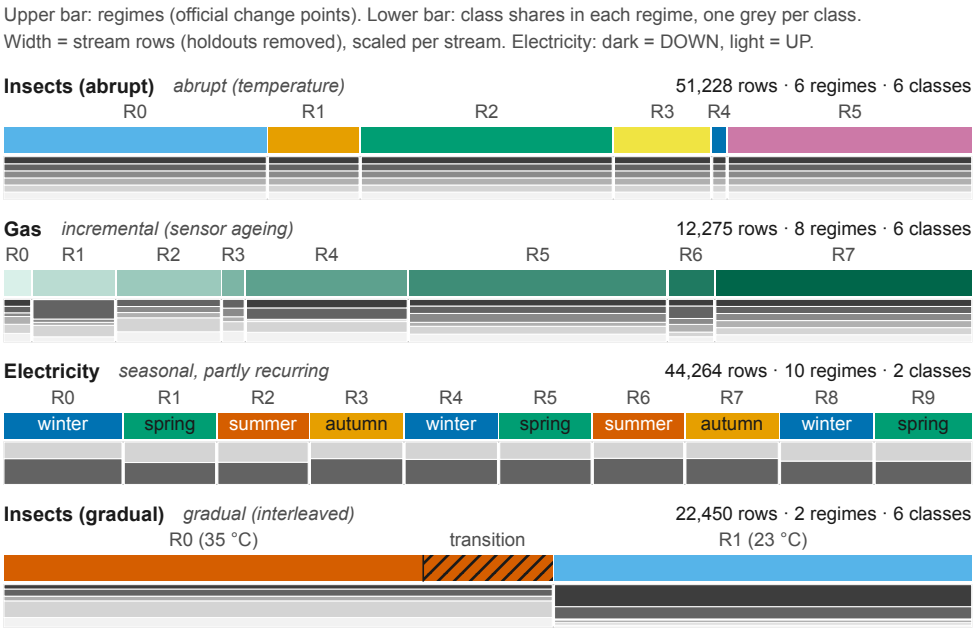


Figure 4: The four data streams. Upper bar: the regimes, cut at the official change points; positions are stream rows after the holdout rows have been removed, and each stream is scaled to the full width. Lower bar: the share of each class within each regime (one grey per class). Class shares are equal in every regime on Insects (abrupt), but each of its change points is preceded by a long run of a single class (Appendix B); they vary between regimes on Gas, stay roughly constant on Electricity, and change between R0 and R1 on Insects (gradual). On Gas and Electricity the classes are mixed within each regime, without such runs. Electricity's regimes are seasons that come back each year. The hatched area on Insects (gradual) is the 3,000-row transition, which belongs to R0.

Table 2: The four data streams (rows, classes, regimes and drift type are shown in Figure 4). Holdout rows are drawn from every regime, removed from the stream and used only to measure retention.

Stream	Regimes and label	Features	Holdout rows	Source
Insects (abrupt)	INSECTS balanced abrupt stream; regimes cut at its five documented changes of temperature	33	1,620	[50]
Gas	Gas Sensor Array Drift (UCI 224); ten time-ordered batches over 36 months, batches of fewer than 300 rows merged into the next (4 with 5, 8 with 9)	128	1,635	[52]
Electricity	New South Wales market (OpenML 151); regimes are seasons; label: price above its mean over the previous 24 hours	4	1,000	[18]
Insects (gradual)	23,000-row slice of the INSECTS imbalanced incremental-gradual stream; R0 is 35 °C ending in a 3,000-row transition, R1 is 23 °C	33	550	[50]

are given in Appendix B. The order in which the classes arrive matters for RQ2: where they are mixed throughout the stream, a class-balanced memory has no reason to keep old rows. The three streams added to Insects (abrupt) met criteria fixed in the pre-registration: regimes definable from documented or natural

time boundaries, enough rows of most classes in every regime for a holdout set, later regimes that differ from earlier ones, a size within TabPFN's limits, and no columns that leak the label. Every stream that met them is used, and none was dropped after results were seen (Appendix B).

3.5 Experimental Design and Statistical Analysis

After the preliminary study (round 1), three rounds ran on one laptop GPU (NVIDIA RTX 2060 Max-Q, 6 GB), with ten seeds per setting. Round 2 compared 16 policies on Insects (abrupt), among them sliding windows of 100 to 2000 rows and the registered archive (Appendix D); round 3 the three policies at both budgets on all four streams (240 new runs, including the boundary variants on the gradual stream, and 40 reused from round 2); round 4 both dual memories with short-term shares of 0.65 and 0.85 at 200 rows on all streams (160 runs; 0.75 comes from round 3).

We report means and standard deviations (SD) over seeds. Two policies are compared through *paired differences* within each seed, so that the seed's effect cancels, with a 95 % t interval, the number of positive seeds and a paired t-test. Within a family of tests, p-values are adjusted by the Holm method [22], which keeps the chance of any false positive in the family below 5 %. A cost in ADAPT is acceptable if the lower end of a one-sided 95 % interval of the difference lies above the *non-inferiority margin* of -0.5 pp.

The hypotheses of round 2 are confirmatory, with fixed thresholds and one-sided, Holm-corrected primary tests; several compare the 1000-row memories with the 200-row sliding window (the pre-registered baseline, one fifth of the budget) (Appendix C). In round 3 the eight comparisons of RET between the regime-balanced dual memory and the class-balanced dual memory form a pre-registered, two-sided, Holm-corrected family without success thresholds. Round 4 is exploratory. Every dual memory is also replayed from the labels alone and must reproduce its fingerprint at every checkpoint, the CPU pilot was reproduced on the GPU, and unit tests cover the policies. As to research integrity, the claims can thus be checked: the hypotheses were fixed before the main runs and changed only by documented amendments (Appendix C), the failed hypotheses and the negative preliminary study are reported (Section 4.3, Appendix A), and code and per-seed results are public (Appendix B).

In short. We run a frozen TabPFN on four public drift streams, compare three memories at two context sizes with ten seeds each, and score each memory by ADAPT and RET with paired, Holm-corrected tests; the hypotheses were fixed before the main runs.

4 RESULTS

This section reports what the experiments found, one research question at a time: how much the model forgets, why the existing memory forgets, and whether our memory forgets less, and at what cost; an exploratory analysis ends it. Two checks come first. The frozen model does learn every stream: on each stream even the lowest ADAPT of the three policies lies far above the better of two trivial baselines that use the same revealed labels (on Insects (abrupt) 74.76 against 31.36 %; the smallest margin is on Electricity, 77.17 against 54.01 %; Appendix B). And the runs are sound: no context held a future row or exceeded its budget, and replays reproduced every dual memory at every checkpoint (Appendix B).

4.1 RQ1: Retention Differs Much More Than Stream Accuracy

Figure 5 draws the means of Table 3 on one scale, so that the contrast is visible; the table gives the exact values. Within each combination of stream and context size, the RET values of the three policies span

Table 3: Adaptation (ADAPT) and retention (RET) of the three context memories on the four streams: balanced accuracy in %, mean over ten seeds. Size: context size in rows. Window: sliding window of the same size; Class-bal.: class-balanced dual memory (Lourenço et al. [35]); Ours: regime-balanced dual memory. The best RET of each row is in bold. Overlap: share of the rows held by the class-balanced dual memory that the window of the same size would also hold (%, mean once the memory is full).

		ADAPT (%)			RET (%)			Overlap
Stream	Size	Window	Class-bal.	Ours	Window	Class-bal.	Ours	Class-bal.
Insects (abrupt)	200	75.68	74.76	74.76	35.08	42.72	**44.17**	91.74
	1000	76.99	76.65	76.84	39.76	45.86	**47.27**	92.61
Gas	200	98.54	98.55	98.64	78.67	78.85	**96.85**	94.67
	1000	99.02	99.03	99.09	81.74	82.02	**97.76**	95.59
Electricity	200	82.33	82.58	80.48	64.48	64.48	**68.39**	95.95
	1000	77.43	77.68	77.17	65.68	65.38	**68.59**	97.49
Insects (gradual)	200	68.80	72.27	71.88	27.52	25.74	**36.89**	90.35
	1000	74.48	76.25	76.01	31.89	31.36	**46.99**	91.05

The standard deviation over the ten seeds is at most 0.54 for ADAPT and 1.38 for RET; the values of every seed are in `results/round3_per_seed.csv` of the public repository (https://github.com/wenboxia/tabular-fm-temporal-shifts). On Insects (gradual), regime-balanced dual memory closes R0 at the documented change point (Appendix B).

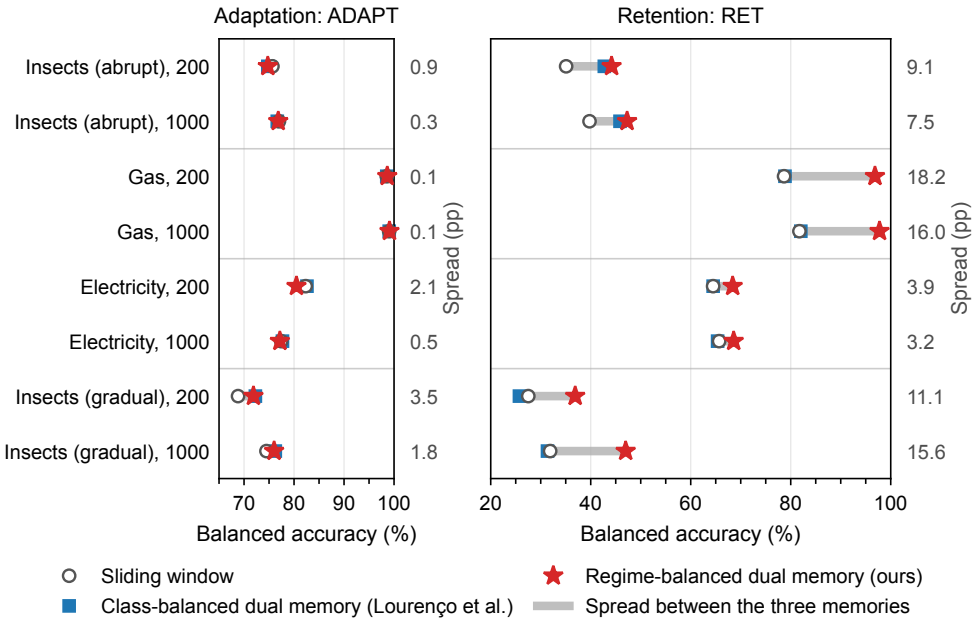


Figure 5: ADAPT (left) and RET (right) of the three memories, the means of Table 3. Both panels use the same scale, so equal distances mean equal differences. The grey bar and the number on the right give the spread between the best and the worst memory in each row: at most 3.5 pp in adaptation, but up to 18.2 pp in retention.

3.21 to 18.18 pp, the ADAPT values only 0.07 to 3.47 pp. On Gas, for example, every policy adapts at 98.54 to 99.09 %, close to the ceiling, while RET ranges from 78.67 to 97.76 %.

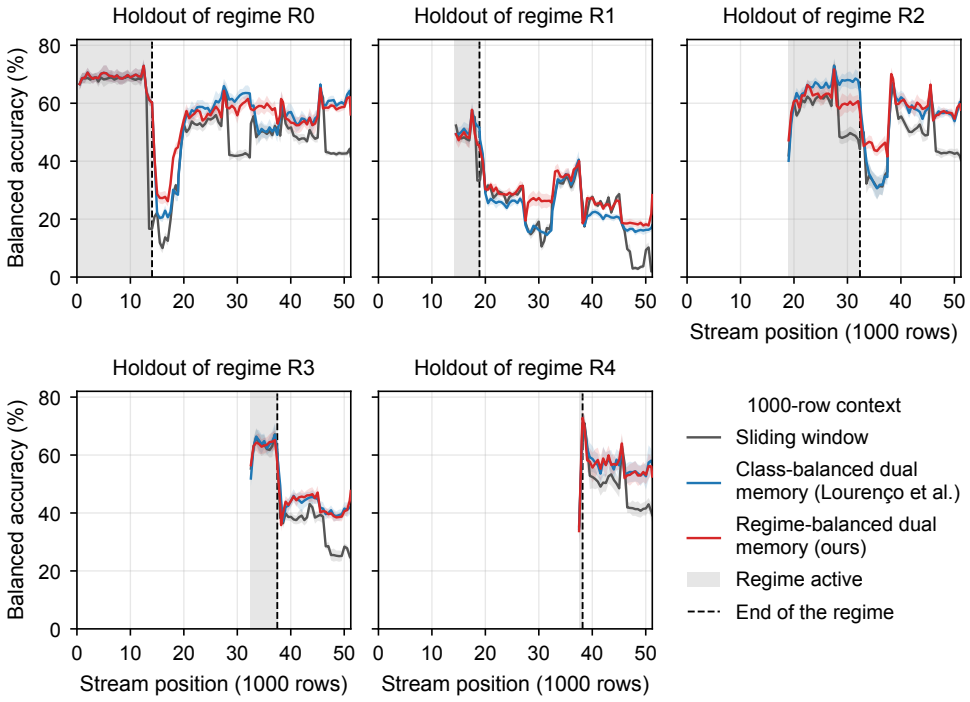


Figure 6: Accuracy on the holdout set of each earlier regime as the Insects (abrupt) stream goes on (1000-row context; mean ± SD, ten seeds). Grey shading: the regime is active; dashed line: its end. After a regime ends, the sliding window loses much of it; at the end of the stream it lies below both dual memories on every regime, on R1 even below chance (16.7 %). While R1 and R2 are active, the regime-balanced dual memory lies 2.37 and 3.73 pp below the class-balanced dual memory on their holdouts (and 1.80 pp while R5 is active; 10/10 each).

The frozen model forgets much of what it has seen. With a sliding window, its accuracy on earlier regimes is far below that on the stream: 35.08 against 75.68 % on Insects (abrupt) at 200 rows, where random guessing gives 16.7 %. Figure 6 shows how this happens over time: once a regime has ended and its rows leave the window, accuracy on it drops. Higher retention did not show up as higher accuracy on the stream: where RET rises most, on Gas, ADAPT changes by at most 0.10 pp, and on Electricity, the only stream whose regimes partly recur, it falls (Section 5.3).

Answer to RQ1. With a sliding window, the frozen model forgets much of each earlier regime once it has ended. The policy changes retention by up to 18.2 pp but adaptation by at most 3.5 pp, so accuracy on the stream alone does not reveal how much a policy forgets.

4.2 RQ2: Class-Balanced Memory Collapses to a Recency Window on Two Streams

Figure 7 shows the collapse on Gas; the eviction rule explains it. The class-balanced dual memory evicts the oldest row of the class with most rows in its long-term part. When the classes arrive mixed, without long runs of a single class, the oldest row of the largest class is nearly always among the oldest rows overall, so the long-term part acts as a queue that extends the short-term one, and an earlier regime leaves the memory soon after its end. This holds on Gas even though its class shares vary between regimes (Figure 4), because within each regime the classes are mixed.

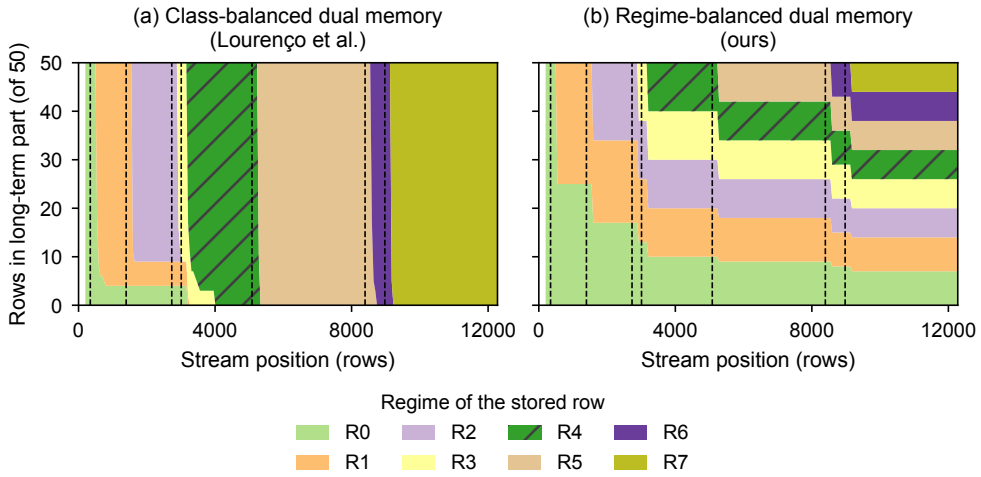


Figure 7: What the 50-row long-term part holds over the Gas stream (200-row context; one run, seed 0). Colours: regime of each stored row; dashed lines: ends of regimes. (a) The class-balanced dual memory (Lourenço et al. [35]): after every change the long-term part fills with the newest regime, and at the end all 50 rows come from the last one, R7. (b) The regime-balanced dual memory (ours): the long-term part is shared among all regimes seen so far (6–7 rows each at the end). Changes appear about 150 rows after a boundary, because new rows first pass through the short-term part. Over ten seeds, once a regime has ended, the whole 200-row class-balanced dual memory holds fewer than five of its rows during 83.6–97.9 % of the later updates on Gas (96.7–99.5 % on Electricity); the regime-balanced dual memory never does. The other streams are in Figure 12.

The overlap column of Table 3 measures this. On Gas and Electricity, 94.7–97.5 % of the rows held by the class-balanced dual memory are among those a sliding window of the same size would hold, and its RET differs from that window by only -0.30 to $+0.28$ pp. This is recency collapse in the strict sense.

The other two streams differ (Figure 12 in Appendix D). On Insects (abrupt) the class-balanced dual memory retains clearly more than the window ($+7.64$ and $+6.10$ pp at 200 and 1000 rows, 10/10), although it still evicts every earlier regime. Its class shares are equal in every regime, but a long run of one class precedes each change point (Appendix B); a plausible, untested explanation is that such runs leave old rows of the other classes in the long-term part. On Insects (gradual) its RET is close to the window's (-1.77 and -0.53 pp) for another reason: it replaces R0 soon after the documented change point. Overlap alone does not explain these differences: it is 90–98 % on all four streams.

Answer to RQ2. The class-balanced dual memory forgets earlier regimes because it balances classes, not regimes, and evicts the oldest row within a class. Where the classes arrive mixed, on Gas and Electricity, it reduces to a recency window; on Insects (abrupt), where long single-class runs precede the change points, it keeps old rows longer, and on Insects (gradual) it loses the old regime soon after the change point.

4.3 RQ3: Regime-Balanced Allocation Reduces Forgetting at a Stream-Dependent Cost

Figure 8 summarises the answer: the regime-balanced dual memory retains more than both other memories on every stream and at both budgets, at a cost in adaptation that is near zero except on Electricity.

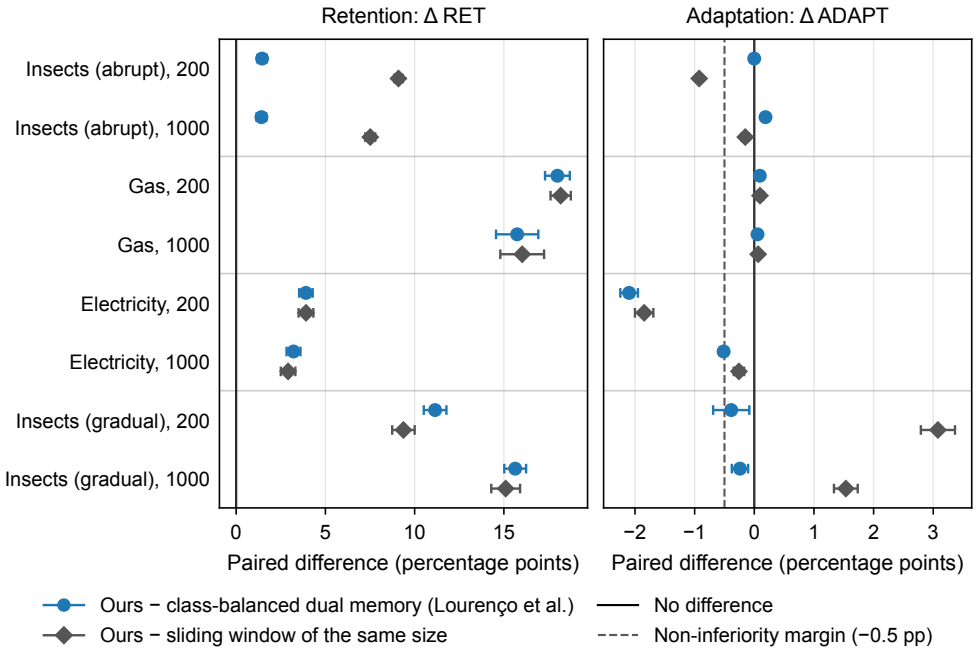


Figure 8: Paired differences between the regime-balanced dual memory and the other two memories at the same context size (ten seeds; markers: mean; bars: two-sided 95 % t intervals). Blue circles: regime-balanced dual memory minus class-balanced dual memory (Lourenço et al. [35]); grey diamonds: regime-balanced dual memory minus sliding window. Left: RET; right: ADAPT, on a different scale. Dashed line: the non-inferiority margin of -0.5 pp (the verdicts use one-sided bounds, which agree with the plotted intervals).

Retention. Relative to the class-balanced dual memory, RET is higher in all eight pre-registered comparisons, in all ten seeds of each and significantly after Holm correction (adjusted $p \leq 1.1 \times 10^{-6}$)[2], by $+1.42$ pp (Insects (abrupt), 1000 rows) to $+18.00$ pp (Gas, 200 rows). Relative to the window of the same size, the gains are $+2.91$ to $+18.18$ pp (10/10 in each). Section 5.1 shows in which regimes they lie.

Cost. Relative to the class-balanced dual memory, the cost in ADAPT is within the margin on Gas and Insects (abrupt), just beyond it on Insects (gradual) at 200 rows (lower bound -0.63 pp) and largest on Electricity (-2.10 pp at 200 rows). Relative to the window, the regime-balanced dual memory adapts worse on Insects (abrupt) and Electricity, in no seed better; on Insects (abrupt) at 200 rows the class-balanced dual memory is just as far below the window, so that cost comes from the dual structure. Because ADAPT averages over the stream, it also hides a cost on the current regime (Figure 6).

No window size buys both. Could a window of another size match the regime-balanced dual memory (Figure 9)? From 100 to 2000 rows, RET rises from 33.81 to 42.11 %, but ADAPT peaks at 600 rows and then falls; as pre-registered (H5), no window size is at least as good as the 1000-row regime-balanced dual memory in both metrics.

[2]The p-values cover only the randomness that differs between seeds, the sampling of the holdout sets and the random state of TabPFN; each stream has a single fixed order (Section 5.5).

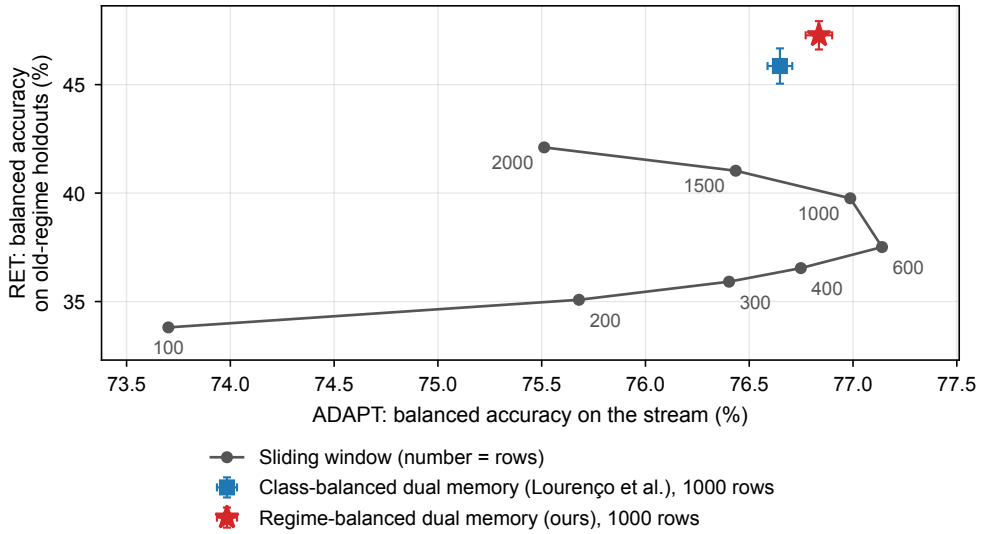


Figure 9: ADAPT and RET on Insects (abrupt) (mean ± SD, ten seeds). The grey line joins sliding windows of 100 to 2000 rows: a larger window retains more, but its adaptation peaks at 600 rows and then falls. No window size reaches the 1000-row regime-balanced dual memory in both metrics. All 16 policies of round 2 are in Figure 13.

Without given boundaries. With regimes closed by ADWIN alarms, the gains shrink but remain (Figure 13; Appendix B, arm (c)). On Insects (abrupt) at 1000 rows the boundary-free variant retains 6.69 pp more than the window (10/10) and 0.59 pp more than the class-balanced dual memory (9/10), with no cost in ADAPT beyond the margin. On Insects (gradual) it retains 9.15 and 11.68 pp more than the class-balanced dual memory (10/10), but its RET varies strongly between seeds at 200 rows, and at 1000 rows it adapts worse than the class-balanced dual memory, beyond the margin.

Pre-registered hypotheses. Of the eight hypotheses of round 2, five hold and three fail (Table 5): H3, because the RET gain over the class-balanced dual memory, +1.42 pp, is below the required 2 pp; H4, because the regime-balanced dual memory recovered more slowly after a change than the 200-row sliding window (the pre-registered baseline, one fifth of the budget) (Section 5.3); and H6, because its gain over the most recent rows of each class is below 2 pp. Several hypotheses compare with the 200-row sliding window (the pre-registered baseline, one fifth of the budget); against the 1000-row window the regime-balanced dual memory changes ADAPT by only −0.15 pp, so its ADAPT gain in H2 comes with the larger budget.

Answer to RQ3. Given the regime boundaries, allocating the long-term part by regime reduces forgetting on all four streams and at both budgets, by +1.42 to +18.00 pp over the class-balanced dual memory. The cost in adaptation depends on the stream: near zero on Gas and Insects (abrupt), small on Insects (gradual) and largest on Electricity (−2.10 pp at 200 rows). Without given boundaries the gains remain but are smaller and, on the gradual stream, less stable.

4.4 Exploratory Analysis: Shares of the Short- and Long-Term Parts

Round 4 varied the short-term share of the two 200-row dual memories (0.65, 0.75 and 0.85, that is 70, 50 and 30 long-term rows), as an exploratory analysis without tests or thresholds (Figure 15 in Appendix D). The RET gain of the regime-balanced dual memory over the class-balanced dual memory keeps its sign in all twelve combinations of stream and share, each 10/10, but its size depends on the share: on Gas and Electricity it shrinks with the long-term part (on Gas from $+18.21$ to $+14.14$ pp), clearly only from 0.75 to 0.85; on Insects (gradual) it is not monotonic; on Insects (abrupt) it hardly changes. On Electricity no share brings the cost in ADAPT within the margin (-2.21 to -1.32 pp).

In short. (1) The choice of memory barely changes accuracy on the stream (by at most 3.5 pp) but changes retention by up to 18.2 pp. (2) On Gas and Electricity the class-balanced dual memory keeps almost only the most recent rows. (3) Given the regime boundaries, the regime-balanced dual memory retains more on all four streams, by $+1.42$ to $+18.00$ pp over the class-balanced dual memory (10/10 runs each), at a cost in adaptation that is largest on Electricity (-2.10 pp). (4) In an exploratory analysis at 200 rows, the gain keeps its sign at all three short-term shares tried (10/10 each).

5 DISCUSSION

This section explains why the results came out as they did, when the regime-balanced dual memory is worth its cost and what this means for sustainability, when keeping the past matters at all and what this means ethically, how the findings fit earlier work, and what limits them.

5.1 Why Regime Balancing Reduces Forgetting

Unlike the class-balanced dual memory (Section 4.2), the regime-balanced dual memory evicts from the regime with most rows: after a change, the older regimes give up rows until the new regime holds the most, and from then on its new rows displace its own older rows. Every ended regime thus keeps a share of the long-term part, however long ago it ended; the shares shrink as regimes accumulate (on Insects (abrupt) at 1000 rows, R1's fell from 125 to 42).

Figure 10 shows where this helps. Small shares can be enough: on Gas at 200 rows, 6 to 12 rows per regime, at the end about one per class, keep the accuracy on every earlier regime at 95–98 %, against 57–91 % for the other two memories. A plausible reading is that sensor ageing moves the features into regions that later data no longer cover: a linear classifier separates each regime from the later ones almost perfectly (cross-validated AUC 0.992–1.000, where 0.5 means inseparable), so a handful of a regime's own rows suffices. On Insects (abrupt) the gain over the class-balanced dual memory lies in R1 and R2 and is near zero on R0, R3 and R4. R1 is the regime least like the others: contexts from other regimes predict it, and contexts from R1 predict them, at only 13–35 % (Figure 14), so only rows of R1 itself help. On Electricity the gain is largest on R1 to R3.

5.2 When Does It Help, and What Does It Cost?

The gain over the class-balanced dual memory is large where that memory keeps few rows of earlier regimes, through collapse (Gas, Electricity) or quick replacement (Insects (gradual)), and small where it already retains them (Insects (abrupt)). It may also grow with how different earlier regimes are from later data: the mean separability of the regimes orders the four streams as the gains do (AUC 0.70 on Insects (abrupt), 0.74 on Electricity, 0.81 on Insects (gradual), 1.00 on Gas), but this is only a conjecture (Section 5.5).

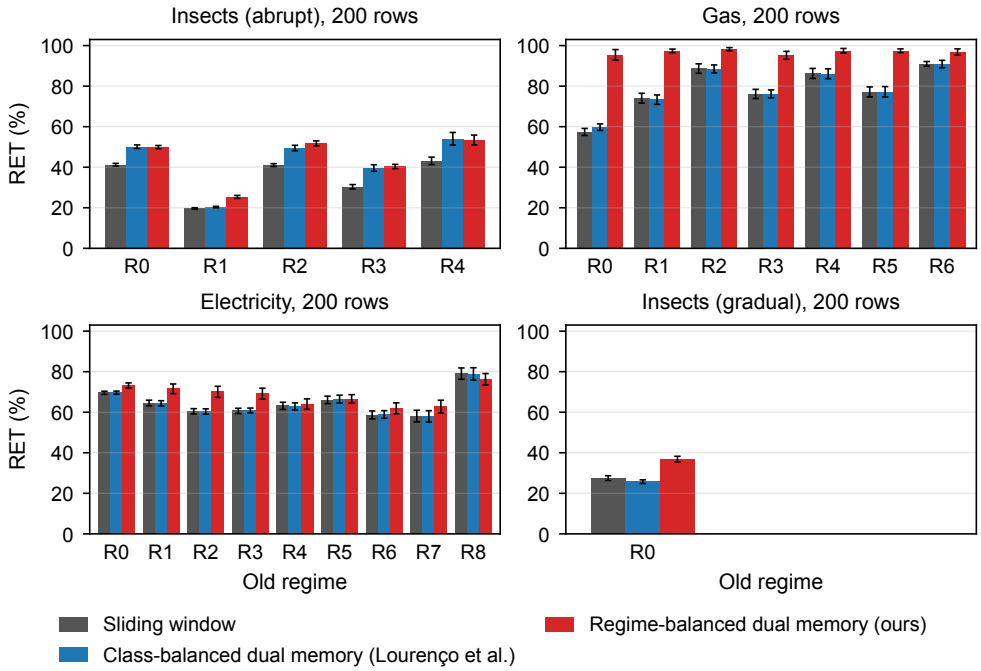


Figure 10: Retention of each earlier regime at 200 context rows (mean ± SD, ten seeds). Grey: sliding window; blue: class-balanced dual memory (Lourenço et al. [35]); red: regime-balanced dual memory (ours). On Insects (gradual) only R0 has ended. Regimes near the end of a stream are measured only briefly after they end.

The cost depends on whether old rows help or mislead predictions on the current regime. It is largest on Electricity, whose label states whether the price is above its mean over the previous 24 hours, a quantity not among the features; older rows reflect other price levels, and even the 1000-row window adapts 4.90 pp worse than the 200-row one. At 200 rows, the median age of the long-term rows is 176 rows (about 3.7 days) in the class-balanced dual memory but 8,448 rows (about 176 days) in the regime-balanced dual memory, and the cost of -2.10 pp is spread over the whole stream rather than concentrated after changes; at 1000 rows, where the long-term rows of the class-balanced dual memory are already about 18 days old, it falls to -0.51 pp. Lourenço et al. [35] likewise found their dual memory slightly, not significantly, below its short-term part alone on Electricity, likely, they wrote, due to strong temporal dependencies.

Sustainability. The choice of memory also matters for the three dimensions of sustainable development. Economically, a frozen model needs no retraining when the data change, so adaptation costs only prediction, which grows with the budget; a small budget can suffice (on Gas the regime-balanced dual memory retained 96.85 % with 200 rows and 97.76 % with 1000). Ecologically, the GPU experiments ran for about 177 hours summed over runs (energy use was not measured) and neither pre-trained nor fine-tuned a foundation model; the smallest budget that gives the required retention is preferable. Socially, a memory that keeps earlier regimes is more dependable when conditions return, as in energy and environmental monitoring, and as a set of rows it is easier to audit than knowledge stored in weights.

5.3 When Is Retention Useful?

Our streams contain no controlled recurrence, so accuracy on the stream could not show the benefit of retention (Section 1.2), and it did not.

Once its labels are revealed, the frozen model learns a returned regime again from the new rows, and a small recent context does this fastest. The pre-registered expectation that kept regimes speed up recovery (H4) failed: after three of the five Insects (abrupt) change points, the regime-balanced dual memory was 4 to 8 pp below the 200-row sliding window (the pre-registered baseline, one fifth of the budget) over the next 500 rows. This is mainly an effect of context size, as the 1000-row window lags the 200-row one as much or more, in line with Verma et al. [53], who find the recovery time of a FIFO memory nearly proportional to its size (their Appendix C.5). Retention is therefore beneficial when an earlier regime must be predicted before its new labels arrive, and detrimental when the target depends only on recent history.

Ethical implications. Forgetting and keeping both raise ethical questions. A memory that keeps only recent rows may serve rare or returning conditions worse, unseen in the accuracy on the stream; if regimes corresponded to groups of people, the forgetting would affect them unequally. Our streams contain no such groups, so this is a possibility, not a finding. Keeping old rows is not harmless either: they can encode relations that no longer hold, as on Electricity, and where rows describe people, keeping them longer conflicts with the principle that personal data be kept no longer than necessary (EU General Data Protection Regulation, Regulation (EU) 2016/679, Art. 5(1)(e)). Which part of the past a model keeps should be a documented decision, not a side effect of the memory policy.

5.4 Relation to Existing Work

Verma et al. [53] found keeping only the most recent rows as effective as the dual memory in prequential accuracy. Recency collapse explains this where the classes arrive mixed: the class-balanced store then holds the most recent rows, so it predicts like a window and, as RET shows, also forgets like one. On their recurring stream (Section 2.4), the dual memory may thus simply not have retained.

The long-term store of Lourenço et al. [35] aims to keep recurring classes, not regimes, so the collapse is not a failure of its design goal; it is the measured form of the authors' remark that the design addresses forgetting of classes only [34].

Like streaming continual learning [6], we score both sides, but for a learner that forgets only through eviction. Recency collapse is the context-level form of the warning that windowing assumes the most recent data to be the relevant ones [12], inherited here by a store designed to balance classes.

Finally, the cost on Electricity mirrors the freshness effect of Wang et al. [54], where a corrector fed with the oldest labels made a frozen tabular model worse on one stream and the most recent labels improved it. Retention shows the other side: rows stale for the current regime are the only evidence the model keeps about earlier ones.

5.5 Limitations

- *Method and metric.* The regime-balanced dual memory uses the documented regime boundaries, and RET averages over exactly these regimes, so the metric is aligned with the method; the other two policies need no boundaries. The boundary-free variant was run only on the two Insects streams and is less stable on the gradual one (Section 4.3). Even there, RET uses the documented

regimes; ADWIN sets only the regimes inside the memory. The method was proposed after the pilot (Section 3.1).

- *Evidence base.* Four streams, one per kind of drift, so the kind of drift is confounded with the label definition, class balance and separability of regimes; one backbone; two budgets. The p-values reflect only holdout sampling and TabPFN's random state: each stream has one order, and the seeds' long-term memories hold nearly the same rows. The 1000-row results on Insects (abrupt) are the round-2 runs, not an independent replication, and were known when the round-3 amendment was written. On Insects (gradual), the rows kept for R0 come from the transition, not from the pure 35 °C part on which R0 is tested.
- *Baselines.* The class-balanced dual memory is the rule of Lourenço et al. [35] scaled to our budgets: 200 rows lie outside their grid of 600 to 1000 rows, and the protocol differs (batches of 50 rows, balanced accuracy, TabPFN-2.5 weights). Not compared: stream learners that are not foundation models, and a long-term store built by class-balanced reservoir sampling, which needs no boundaries. On Electricity, repeating the previous label row by row would score about 85 %, more than any policy here, so those results hold for batches of 50 rows.
- *Interpretation.* RET is a proxy for recurrence; its effect on recovery after a return was not measured. The explanations for Insects (abrupt), Gas and Electricity were written after the results were seen and not tested by ablation. That the gain grows with the difference between regimes is a conjecture: four streams, a measure chosen after the results, and an order that fails against the sliding window and within streams. Round 4 is exploratory, at 200 rows only and with three shares.

In short. Regime balancing works because every ended regime keeps a few of its own rows. It helps most where the class-balanced dual memory collapses or quickly replaces old regimes, costs most where the target depends on the recent past, and should pay off when an earlier regime must be predicted before its new labels arrive (not measured here, as the streams have no controlled recurrence). The evidence comes from four streams and one backbone, and the method needs the regime boundaries.

6 CONCLUSIONS AND FUTURE WORK

Tabular foundation models such as TabPFN are now applied to data that change over time. Their weights hold only general knowledge from pre-training, so the context is the model's only memory of the stream, and a bounded context must decide what to forget. This matters because earlier situations can return, yet the usual score, accuracy on the incoming stream, cannot show what is forgotten. We therefore measured both adaptation and retention for three context memories on four public streams, in a pre-registered study. Giving every past regime a share of the long-term memory kept the past better on every stream, by 1.4 to 18.0 points, at a cost that depends on the stream. On Gas, for example, accuracy on earlier regimes rose from 78.85 to 96.85 % with 200 context rows, while accuracy on the stream stayed at about 98.6 %.

Main findings.

- *RQ1, how much is forgotten?* Much, and stream accuracy hides it: the three memories differ by at most 3.5 pp on the stream but by up to 18.2 pp on earlier regimes. With only recent rows, the model scores 35.08 % on earlier regimes of Insects (abrupt) at 200 rows, about twice chance (16.7 % for six classes), while it scores 75.68 % on the stream.

- *RQ2, why does an existing memory forget?* The class-balanced dual memory balances classes, not regimes; where the classes arrive mixed, on Gas and Electricity, it keeps almost only the most recent rows (94.7–97.5 % overlap with a sliding window, retention within 0.3 pp of it).
- *RQ3, does balancing by regime help, and at what cost?* Yes: given the regime boundaries, the regime-balanced dual memory retains more than the class-balanced dual memory on all four streams and at both budgets, by $+1.42$ to $+18.00$ pp (10/10 runs each); the gain is smallest on Insects (abrupt) but positive in every run. The cost in adaptation ranges from near zero (Gas, Insects (abrupt)) to -2.10 pp (Electricity). With boundaries from a drift detector, tested only on the two Insects streams, the gains remain but are smaller and, on the gradual stream, less stable.

Contributions and novelty.

(1) *Making forgetting visible:* retention on held-out data of earlier regimes, scored next to stream accuracy, for a frozen tabular foundation model with a fixed context; the studies of such models on streams that we found report only stream accuracy.
(2) *Explaining why an existing memory forgets:* the class-balanced memory proposed for TabPFN quietly becomes "keep the newest rows" when the classes arrive mixed (recency collapse, on Gas and Electricity).
(3) *A simple remedy:* give every past regime its share of the long-term context. The idea comes from replay memories in continual learning [4, 31]; what is new is applying it to a frozen model's context, with drift regimes as the groups.

Implications for practice. The results concern those who apply or evaluate tabular foundation models on changing data, as in sensor, energy or environmental monitoring. Balancing the long-term memory by regime is worthwhile when earlier situations can return, their boundaries are known (such as seasons or sensor batches) or, tentatively, can be detected (tested on the two Insects streams only), and the target does not depend only on the recent past; otherwise, as on Electricity, a sliding window is better. Evaluations should report retention alongside accuracy on the stream.

Future work. In order of importance: (1) Streams in which earlier situations really return, to measure whether the kept rows make the model recover faster; so far RET is only a proxy for this. (2) Removing the need for given boundaries, the main practical limit: run the detector-based variant on Gas and Electricity, and compare with class-balanced reservoir sampling [5], which needs no boundaries. (3) Checking how far the findings generalise: other backbones (TabICL [47], the original TabPFN of Lourenço et al. [35]), more streams per kind of drift, and regression. (4) A short/long split that adapts to the stream, to reduce the cost on Electricity.

Take-home message. For a frozen tabular foundation model, the rows in its context are all it remembers of the stream: keep a few rows from every past regime when the past may return, and report retention next to stream accuracy.

ACKNOWLEDGMENTS

I am deeply grateful to my supervisor, Tianyu Wang, for his careful guidance throughout this degree project: for pointing out a promising direction and for his expert advice at every stage. I also thank KTH Royal Institute of Technology for my years of study here. The teachers of the courses I took were of a

very high standard, and what I learned from them gave me the solid foundation on which this project was built.

REFERENCES

[1] Alif Ashrafee, Jedrzej Kozal, Michal Wozniak, and Bartosz Krawczyk. 2025. Holistic Continual Learning under Concept Drift with Adaptive Memory Realignment. arXiv:2507.02310 [cs.LG] https://arxiv.org/abs/2507.02310

[2] Albert Bifet and Ricard Gavaldà. 2007. Learning from Time-Changing Data with Adaptive Windowing. In *Proceedings of the 2007 SIAM International Conference on Data Mining*. Society for Industrial and Applied Mathematics, 443–448. https://doi.org/10.1137/1.9781611972771.42

[3] Kay Henning Brodersen, Cheng Soon Ong, Klaas Enno Stephan, and Joachim M. Buhmann. 2010. The Balanced Accuracy and Its Posterior Distribution. In *2010 20th International Conference on Pattern Recognition*. IEEE, 3121–3124. https://doi.org/10.1109/ICPR.2010.764

[4] Arslan Chaudhry, Marcus Rohrbach, Mohamed Elhoseiny, Thalaiyasingam Ajanthan, Puneet K. Dokania, Philip H. S. Torr, and Marc'Aurelio Ranzato. 2019. On Tiny Episodic Memories in Continual Learning. arXiv:1902.10486 [cs.LG] https://arxiv.org/abs/1902.10486

[5] Aristotelis Chrysakis and Marie-Francine Moens. 2020. Online Continual Learning from Imbalanced Data. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*, Hal Daumé III and Aarti Singh (Eds.). PMLR, 1952–1961. https://proceedings.mlr.press/v119/chrysakis20a.html

[6] Andrea Cossu, Federico Giannini, Giacomo Ziffer, Alessio Bernardo, Alexander Gepperth, Emanuele Della Valle, Barbara Hammer, and Davide Bacciu. 2026. A practical guide to streaming continual learning. *Neurocomputing* 674 (April 2026), 132951. https://doi.org/10.1016/j.neucom.2026.132951

[7] Vitor Crista, Afonso Lourenço, Diogo Martinho, and Goreti Marreiros. 2026. Streaming Hierarchical Inference with Tabular Foundation Models. arXiv:2609.07956 [cs.LG] https://arxiv.org/abs/2609.07956

[8] Benjamin Feuer, Robin Schirrmeister, Valeriia Cherepanova, Chinmay Hegde, Frank Hutter, Micah Goldblum, Niv Cohen, and Colin White. 2024. TuneTables: Context Optimization for Scalable Prior-Data Fitted Networks. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 83430–83464. https://doi.org/10.52202/079017-2654

[9] R French. 1999. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences* 3, 4 (April 1999), 128–135. https://doi.org/10.1016/s1364-6613(99)01294-2

[10] João Gama and Petr Kosina. 2014. Recurrent concepts in data streams classification. *Knowledge and Information Systems* 40, 3 (2014), 489–507. https://doi.org/10.1007/s10115-013-0654-6

[11] João Gama, Raquel Sebastião, and Pedro Pereira Rodrigues. 2013. On evaluating stream learning algorithms. *Machine Learning* 90, 3 (2013), 317–346. https://doi.org/10.1007/s10994-012-5320-9

[12] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. 2014. A survey on concept drift adaptation. *Comput. Surveys* 46, 4 (March 2014), 1–37. https://doi.org/10.1145/2523813

[13] Heitor M. Gomes, Albert Bifet, Jesse Read, Jean Paul Barddal, Fabrício Enembreck, Bernhard Pfahringer, Geoff Holmes, and Talel Abdessalem. 2017. Adaptive random forests for evolving data stream classification. *Machine Learning* 106, 9-10 (June 2017), 1469–1495. https://doi.org/10.1007/s10994-017-5642-8

[14] Paulo Mauricio Gonçalves Jr and Roberto Souto Maior de Barros. 2013. RCD: A recurring concept drift framework. *Pattern Recognition Letters* 34, 9 (July 2013), 1018–1025. https://doi.org/10.1016/j.patrec.2013.02.005

[15] Léo Grinsztajn, Klemens Flöge, Oscar Key, Felix Birkel, Philipp Jund, Brendan Roof, Benjamin Jäger, Dominik Safaric, Simone Alessi, Adrian Hayler, Mihir Manium, Rosen Yu, Felix Jablonski, Shi Bin Hoo, Anurag Garg, Jake Robertson, Magnus Bühler, Vladyslav Moroshan, Lennart Purucker, Clara Cornu, Lilly Charlotte Wehrhahn, Alessandro Bonetto, Bernhard Schölkopf, Sauraj Gambhir, Noah Hollmann, and Frank Hutter. 2026. TabPFN-2.5: Advancing the State of the Art in Tabular Foundation Models. arXiv:2511.08667 [cs.LG] https://arxiv.org/abs/2511.08667

[16] Stephen Grossberg. 1980. How does a brain build a cognitive code? *Psychological Review* 87, 1 (1980), 1–51. https://doi.org/10.1037/0033-295x.87.1.1

[17] Nuwan Gunasekara, Bernhard Pfahringer, Heitor Murilo Gomes, and Albert Bifet. 2023. Survey on Online Streaming Continual Learning. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI-2023)*. International Joint Conferences on Artificial Intelligence Organization, 6628–6637. https://doi.org/10.24963/ijcai.2023/743

[18] Michael Harries. 1999. *Splice-2 Comparative Evaluation: Electricity Pricing*. Technical Report. University of New South Wales.

[19] Kai Helli, David Schnurr, Noah Hollmann, Samuel Müller, and Frank Hutter. 2024. Drift-Resilient TabPFN: In-Context Learning Temporal Distribution Shifts on Tabular Data. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 98742–98781. https://doi.org/10.52202/079017-3134

[20] Noah Hollmann, Samuel Müller, Katharina Eggensperger, and Frank Hutter. 2023. TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second. In *International Conference on Learning Representations*. https://mlanthology.org/iclr/2023/hollmann2023iclr-tabpfn/

[21] Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter. 2025. Accurate predictions on small data with a tabular foundation model. *Nature* 637, 8045 (Jan. 2025), 319–326. https://doi.org/10.1038/s41586-024-08328-6

[22] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70. https://www.jstor.org/stable/4615733

[23] Liuwang Kang, Fan Wang, Shaoshan Liu, Hung-Chyun Chou, Chuan Lin, and Ning Ding. 2025. In-Context Learning can Perform Continual Learning Like Humans. arXiv:2509.22764 [cs.LG] https://arxiv.org/abs/2509.22764

[24] Ioannis Katakis, Grigorios Tsoumakas, and Ioannis Vlahavas. 2010. Tracking recurring contexts using ensemble classifiers: an application to email filtering. *Knowledge and Information Systems* 22, 3 (2010), 371–391. https://doi.org/10.1007/s10115-009-0206-2

[25] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. 2017. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences* 114, 13 (March 2017), 3521–3526. https://doi.org/10.1073/pnas.1611835114

[26] Lukasz Korycki and Bartosz Krawczyk. 2021. Class-Incremental Experience Replay for Continual Learning under Concept Drift. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. IEEE, 3644–3653. https://doi.org/10.1109/cvprw53098.2021.00404

[27] Jinmo Lee, Doyun Choi, Moongi Choi, and Jaemin Yoo. 2026. Bounded Context Management for Tabular Foundation Models on Stream Learning. arXiv:2606.18677 [cs.LG] https://arxiv.org/abs/2606.18677

[28] Guangyu Li, Meng Ding, and Lijie Hu. 2026. Understanding Generalization and Forgetting in In-Context Continual Learning. arXiv:2605.28705 [cs.LG] https://arxiv.org/abs/2605.28705 Accepted at ICML 2026.

[29] Jiachang Liu, Dinghan Shen, Yizhe Zhang, Bill Dolan, Lawrence Carin, and Weizhu Chen. 2022. What Makes Good In-Context Examples for GPT-3?. In *Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures*, Eneko Agirre, Marianna Apidianaki, and Ivan Vulić (Eds.). Association for Computational Linguistics, Dublin, Ireland and Online, 100–114. https://doi.org/10.18653/v1/2022.deelio-1.10

[30] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. https://doi.org/10.1162/tacl_a_00638

[31] David Lopez-Paz and Marc'Aurelio Ranzato. 2017. Gradient Episodic Memory for Continual Learning. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc., 6467–6476. https://proceedings.neurips.cc/paper_files/paper/2017/file/f87522788a2be2d171666752f97ddebb-Paper.pdf

[32] Viktor Losing, Barbara Hammer, and Heiko Wersing. 2016. KNN Classifier with Self Adjusting Memory for Heterogeneous Concept Drift. In *2016 IEEE 16th International Conference on Data Mining (ICDM)*. IEEE, 291–300. https://doi.org/10.1109/icdm.2016.0040

[33] Viktor Losing, Barbara Hammer, and Heiko Wersing. 2017. Tackling heterogeneous concept drift with the Self-Adjusting Memory (SAM). *Knowledge and Information Systems* 54, 1 (Dec. 2017), 171–201. https://doi.org/10.1007/s10115-017-1137-y

[34] Afonso Lourenço, João Gama, Eric P. Xing, and Goreti Marreiros. 2025. Bridging Streaming Continual Learning via In-Context Large Tabular Models. arXiv:2512.11668 [cs.LG] https://arxiv.org/abs/2512.11668

[35] Afonso Lourenço, João Gama, Eric P. Xing, and Goreti Marreiros. 2026. In-context Learning of Evolving Data Streams with Tabular Foundational Models. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1*. ACM, 973–982. https://doi.org/10.1145/3770854.3780305

[36] Jie Lu, Anjin Liu, Fan Dong, Feng Gu, Joao Gama, and Guangquan Zhang. 2019. Learning under Concept Drift: A Review. *IEEE Transactions on Knowledge and Data Engineering* 31, 12 (Dec. 2019), 2346–2363. https://doi.org/10.1109/tkde.2018.2876857

[37] Yao Lu, Max Bartolo, Alastair Moore, Sebastian Riedel, and Pontus Stenetorp. 2022. Fantastically Ordered Prompts and Where to Find Them: Overcoming Few-Shot Prompt Order Sensitivity. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 8086–8098. https://doi.org/10.18653/v1/2022.acl-long.556

[38] Michael McCloskey and Neal J. Cohen. 1989. Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem. In *Psychology of Learning and Motivation*. Elsevier, 109–165. 9780125433242 https://doi.org/10.1016/s0079-7421(08)60536-8

[39] Martial Mermillod, Aurélia Bugaiska, and Patrick Bonin. 2013. The stability-plasticity dilemma: investigating the continuum from catastrophic forgetting to age-limited learning effects. *Frontiers in Psychology* 4 (2013). https://doi.org/10.3389/fpsyg.2013.00504

[40] Sewon Min, Xinxi Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. 2022. Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (Eds.). Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 11048–11064. https://doi.org/10.18653/v1/2022.emnlp-main.759

[41] Saleh Momeni, Sahisnu Mazumder, Zixuan Ke, and Bing Liu. 2025. In-context Continual Learning Assisted by an External Continual Learner. In *Proceedings of the 31st International Conference on Computational Linguistics*, Owen Rambow, Leo Wanner,

Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert (Eds.). Association for Computational Linguistics, Abu Dhabi, UAE, 7292–7306. https://aclanthology.org/2025.coling-main.487/

[42] Jacob Montiel, Max Halford, Saulo Martiello Mastelini, Geoffrey Bolmier, Raphaël Sourty, Robin Vaysse, Adil Zouitine, Heitor Murilo Gomes, Jesse Read, Talel Abdessalem, and Albert Bifet. 2021. River: machine learning for streaming data in Python. *Journal of Machine Learning Research* 22, 110 (2021), 1–8. http://jmlr.org/papers/v22/20-1380.html

[43] Samuel Müller, Noah Hollmann, Sebastian Pineda Arango, Josif Grabocka, and Frank Hutter. 2022. Transformers Can Do Bayesian Inference. In *International Conference on Learning Representations*. https://mlanthology.org/iclr/2022/muller2022iclr-transformers/

[44] Nikita Narayanan, Ritham Majumdarr, and Sonali Parbhoo. 2026. Continual Learning without Continual Training. arXiv:2610.10379 [cs.LG] https://arxiv.org/abs/2610.10379

[45] German I. Parisi, Ronald Kemker, Jose L. Part, Christopher Kanan, and Stefan Wermter. 2019. Continual lifelong learning with neural networks: A review. *Neural Networks* 113 (May 2019), 54–71. https://doi.org/10.1016/j.neunet.2019.01.012

[46] Ameya Prabhu, Philip H. S. Torr, and Puneet K. Dokania. 2020. GDumb: A Simple Approach that Questions Our Progress in Continual Learning. In *Computer Vision – ECCV 2020*. Springer International Publishing, 524–540. 9783030585365 https://doi.org/10.1007/978-3-030-58536-5_31

[47] Jingang Qu, David Holzmüller, Gaël Varoquaux, and Marine Le Morvan. 2025. TabICL: A Tabular Foundation Model for In-Context Learning on Large Data. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 50817–50847. https://mlanthology.org/icml/2025/qu2025icml-tabicl/

[48] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H. Lampert. 2017. iCaRL: Incremental Classifier and Representation Learning. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 5533–5542. https://doi.org/10.1109/cvpr.2017.587

[49] Ohad Rubin, Jonathan Herzig, and Jonathan Berant. 2022. Learning To Retrieve Prompts for In-Context Learning. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Marine Carpuat, Marie-Catherine de Marneffe, and Ivan Vladimir Meza Ruiz (Eds.). Association for Computational Linguistics, Seattle, United States, 2655–2671. https://doi.org/10.18653/v1/2022.naacl-main.191

[50] Vinicius M. A. Souza, Denis M. dos Reis, André G. Maletzke, and Gustavo E. A. P. A. Batista. 2020. Challenges in benchmarking stream learning algorithms with real-world data. *Data Mining and Knowledge Discovery* 34, 6 (July 2020), 1805–1858. https://doi.org/10.1007/s10618-020-00698-5

[51] Valentin Thomas, Junwei Ma, Rasa Hosseinzadeh, Keyvan Golestan, Guangwei Yu, Maksims Volkovs, and Anthony Caterini. 2024. Retrieval & Fine-Tuning for In-Context Tabular Models. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 108439–108467. https://doi.org/10.52202/079017-3442

[52] Alexander Vergara, Shankar Vembu, Tuba Ayhan, Margaret A. Ryan, Margie L. Homer, and Ramón Huerta. 2012. Chemical gas sensor drift compensation using classifier ensembles. *Sensors and Actuators B: Chemical* 166–167 (2012), 320–329. https://doi.org/10.1016/j.snb.2012.01.074

[53] Nilesh Verma, Daniel Nowak-Assis, Afonso Lourenço, Albert Bifet, Bernhard Pfahringer, Maroua Bahri, and Nick Jin Sean Lim. 2026. Rethinking Tabular Foundation Models On Data Streams. arXiv:2610.05352 [cs.LG] https://arxiv.org/abs/2610.05352

[54] Tianyu Wang, Xi Vincent Wang, Lihui Wang, Mian Li, and Zhihao Liu. 2026. The Anatomy and Boundary of Adaptation under Temporal Tabular Shift. arXiv:2609.12136 [stat.ME] https://arxiv.org/abs/2609.12136

[55] Zifeng Wang, Zizhao Zhang, Sayna Ebrahimi, Ruoxi Sun, Han Zhang, Chen-Yu Lee, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer Dy, and Tomas Pfister. 2022. DualPrompt: Complementary Prompting for Rehearsal-Free Continual Learning. In *Computer Vision – ECCV 2022*. Springer Nature Switzerland, 631–648. 9783031198090 https://doi.org/10.1007/978-3-031-19809-0_36

[56] Zifeng Wang, Zizhao Zhang, Chen-Yu Lee, Han Zhang, Ruoxi Sun, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer Dy, and Tomas Pfister. 2022. Learning to Prompt for Continual Learning. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 139–149. https://doi.org/10.1109/cvpr52688.2022.00024

[57] Derek Xu, Olcay Cirit, Reza Asadi, Yizhou Sun, and Wei Wang. 2024. Mixture of In-Context Prompters for Tabular PFNs. arXiv:2405.16156 [cs.LG] https://arxiv.org/abs/2405.16156

[58] Zihao Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. 2021. Calibrate Before Use: Improving Few-shot Performance of Language Models. In *Proceedings of the 38th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 139)*, Marina Meila and Tong Zhang (Eds.). PMLR, 12697–12706. https://proceedings.mlr.press/v139/zhao21c.html

A PRELIMINARY STUDY: THREE-LEVEL SYSTEM AND DRIFT DETECTION

This appendix summarises the preliminary study that preceded the change of direction (Section 3.1).

The three-level system. A frozen TabPFN with a 200-row sliding context gives a slow prediction. A fast corrector (nearest neighbours or an exponential average over the residuals of recently labelled rows)

Table 4: Main results of the preliminary study. Differences are in plain (not balanced) accuracy, in percentage points, relative to TabPFN with a sliding context alone; "sig." marks a paired t-test beyond the critical value ($n = 5$), "n.s." one within it.

Experiment	Result
Fast corrector alone (synthetic)	$+0.04$, $+0.12$, -0.14 on regime switching, rotating boundary and combined drift; none significant.
Full three-level system (synthetic)	$+1.00$ (sig.) on rotating boundary, -0.18 (n.s.) on regime switching, -0.47 (sig.) on combined drift.
Context reset at the true change points (synthetic, regime switching)	$+0.51$ (sig.; better in 5/5 seeds), $+2.36$ on the rows after the changes.
Five variants of detector input and adapter (synthetic)	combined drift -0.28 to -0.55 in all five; rotating boundary $+1.32$ to $+1.52$ (sig.). In the three variants with alarms, all 52 routing decisions created a new adapter; an existing one was never reused.
Three-level system on real streams (CPU)	Electricity -0.06 (n.s.); Insects -0.17 (sig.), on segments that later turned out to cover only two of the five documented change points. The own detector raised no alarm in 20 Insects runs and one in 15 Electricity runs.
Detector replay on the saved error streams	The own ADWIN used a bound without the empirical variance and needed a shift of the error rate of at least 0.209 for a 200/200 split. The river ADWIN ($\delta = 0.002$) on the same errors raised alarms in 16 of 20 Insects runs, detected every covered change point (median delay 266 rows) and stayed nearly silent on Electricity (1 of 15 runs).
Round 1: GPU, Insects, river ADWIN, one seed	Actions after an alarm changed the errors on a segment by -6 to $+5$ (smallest $p = 0.36$; McNemar), also when triggered at the true change points. With no action, the three-level system made 26 and 23 more errors than TabPFN alone on two segments ($p = 0.005$ and 0.001, sign test) and 4 fewer on the third ($p = 0.72$).

and a small gated adapter, one per regime in later versions, add corrections; a gate weights the three outputs. A drift detector (ADWIN on the model's errors) can trigger an action on an alarm: switching to or creating an adapter, resetting the context to its most recent rows, or clearing the corrector's buffer.

Synthetic streams. Three generators were used: *rotating boundary* (two Gaussian features and a linear boundary that rotates slowly; gradual drift), *regime switching* (three regimes with independent feature means and decision weights, cycling every 500 rows; abrupt drift) and *combined drift* (five features shift slowly while the boundary changes abruptly every 3,000 rows).

Results. Table 4 lists the main results. Paired comparisons on synthetic streams use five seeds, each of which generates a different stream; on the real streams the five runs per segment used the same data and differed only in the initialisation of the adapter, so their p-values do not describe independent replications.

Two observations of round 1 shaped the later design. Each change point of the Insects segments came at the end of a run of a single class (34 to 538 rows), and the alarms mostly followed these changes of class proportions rather than the documented change points. And even a perfect action had little room to help: after two of the three change points, the excess errors over the new regime's steady state were below what the test could detect. The event-triggered system thus gave no measurable gain and could not be tested well on these segments, whereas retention of earlier regimes does not depend on the short period after a change. This motivated the study of the context itself.

B DATASET DETAILS AND REPRODUCIBILITY

This appendix gives the source, preprocessing, regimes and holdout sets of each stream (Table 2) and the information needed to reproduce the results. Content hashes of the three streams added in round 3 are in the pre-registration in the public repository.

Insects (abrupt). The balanced abrupt stream of the INSECTS benchmark [50]: 52,848 rows, 33 features, 6 classes. The paper states no licence for the data; we use them as a public research benchmark.

Its documented change points, rows 14,352, 19,500, 33,240, 38,682 and 39,510, define six regimes, R0 to R5 (changes of temperature). Each seed draws 50 rows per class from each regime (20 per class from the short R4), uniformly over the whole regime, as holdout set (1,620 rows); in the remaining stream of 51,228 rows the change points lie at 14,052, 18,900, 32,340, 37,482 and 38,190. In the full data set, before the holdout sets are removed, each change point is preceded by a run of a single class (1,754, 74, 97, 537 and 34 rows), and the data end with one of 822 rows; in the stream these runs are slightly shorter. Adaptation is therefore also reported with all runs of at least 30 identical labels removed (*masked* ADAPT, used in hypothesis H2); the run before the fifth change point reaches 30 rows in only four seeds. The masked values are in the public repository. In the boundary-free variant (Section 3.3, 1000 rows), ADWIN raised 13 to 15 alarms per seed, found the three earliest change points within 11 to 79 rows, and raised its other alarms elsewhere.

Gas. The Gas Sensor Array Drift data of Vergara et al. [52] (UCI dataset 224, also OpenML 1476; licence CC BY 4.0): 13,910 rows of 128 sensor features and 6 gases, recorded in ten batches over 36 months. The source file begins with the last batch, so the batches are put back in time order; each covers one to five recording months. Batches with fewer than 300 rows are merged with the next (batches 4 and 5, 8 and 9), giving eight regimes. The order of rows inside a batch is grouped by label in the source and undocumented, so it is shuffled once with a fixed seed; every run uses the same order. Holdout sets here and in the two following streams take, per regime and class, the smaller of 50 rows and a quarter of the eligible rows, and no rows when this would be fewer than 10; this gives 1,635 rows on Gas.

Electricity. The New South Wales electricity market data described by Harries [18] (OpenML dataset 151, listed there as public), 943 days of 48 half-hours (45,264 rows) after removing day 363, a near-duplicate of day 362. The features are the day of the week, the half-hour, the price and the demand in New South Wales. The date column is dropped because, with labels revealed in batches, it acts as a time index into strongly autocorrelated labels; the three columns for Victoria and the transfer between the states are dropped because they are constant placeholders for the initial 17,424 rows. The label states whether the price is above its mean over the previous 24 hours [18]. The regimes are the meteorological seasons of the southern hemisphere, with partial seasons shorter than 30 days merged into their neighbour: ten regimes, 1,000 holdout rows.

Insects (gradual). A slice of the imbalanced incremental-gradual INSECTS stream around its documented change point: 10,000 rows at 35 °C, a transition of 3,000 rows and 10,000 rows at 23 °C. Matching every row to the temperature-ordered version of the data showed that in the transition instances of both temperatures are interleaved, with about 30, 50 and 70 % at 23 °C in its three thirds. R0 is the 35 °C part with the transition, R1 the 23 °C part; transition rows are not used for holdout sets, and R0 has none for one class with only ten pure rows (550 holdout rows). Since the end of R0 is ambiguous, the memory was closed in three ways: (a) at the documented change point, the main arm used in Sections 4 and 5; (b) at the start of the transition, which is known only from the analysis above and so cannot be used in deployment; and (c) by ADWIN alarms. Evaluation regimes and holdout sets are the same in all arms (Figure 11). The RET of the regime-balanced dual memory was 36.89, 33.17 and 34.89 % in arms (a), (b) and (c) at 200 rows and 46.99, 50.78 and 43.04 % at 1000 rows, all above the class-balanced dual memory (25.74 and 31.36 %); arm (b) is therefore not an upper bound at 200 rows. Arm (c) is the least stable: at 200 rows its RET has an SD of 5.25 pp over the seeds, and at 1000 rows its ADAPT is 0.82 pp

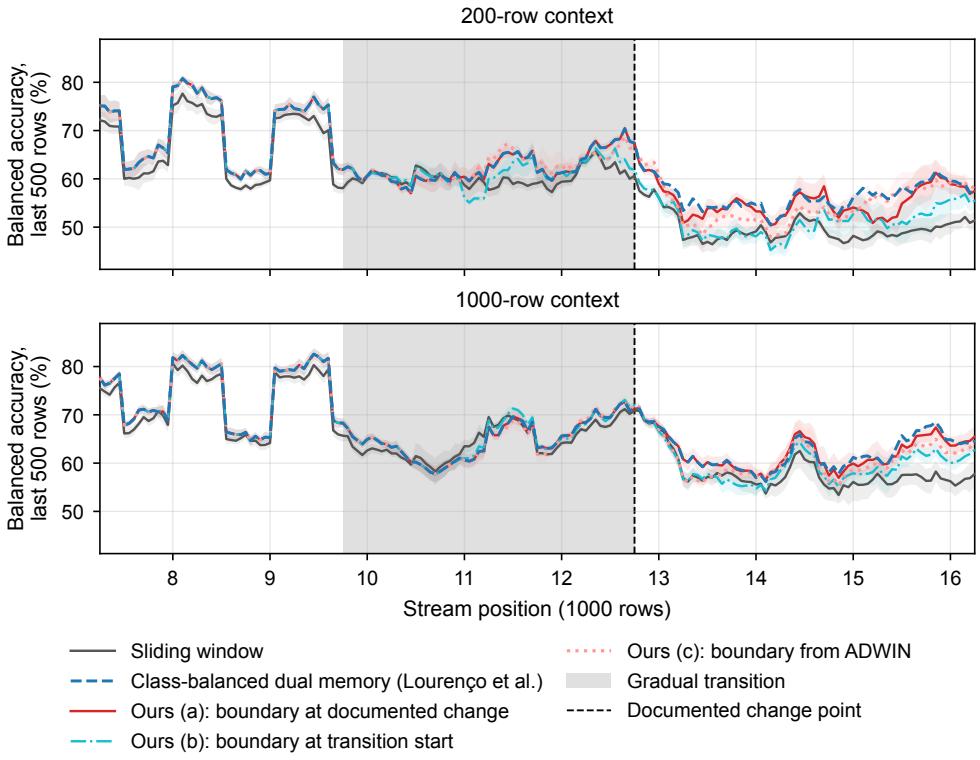


Figure 11: Accuracy around the gradual transition of Insects (gradual), measured over the last 500 rows (mean ± SD, ten seeds; top: 200 rows, bottom: 1000 rows). Grey: the 3,000-row transition in which both temperatures are mixed; dashed: the documented change point. Arms (a)–(c) of the regime-balanced dual memory close R0 at the documented change point, at the start of the transition and at ADWIN alarms. Before the change point, the class-balanced dual memory and arm (a) hold the same rows; afterwards, the sliding window is mostly the lowest curve. Because each curve looks back 500 rows, a change shows up to 500 rows after it happens.

below the class-balanced dual memory (0/10). At the end of the stream, arm (a) keeps rows from the end of the transition for R0 and arm (b) rows of the pure 35 °C part.

Trivial baselines. The two baselines of Section 4 use only the revealed labels: the last label revealed before a batch, and the majority class of the last 200 revealed rows. The lowest ADAPT of the three policies lies far above the better of the two on every stream: 74.76 against 31.36 % on Insects (abrupt), 98.54 against 21.13 % on Gas, 77.17 against 54.01 % on Electricity and 68.80 against 28.78 % on Insects (gradual). The pre-registration defined them row by row, which uses labels of the current batch that the model has not seen; on Electricity, repeating the previous row's label then scores 84.86 % balanced accuracy, because neighbouring labels are strongly correlated. With labels revealed per batch, as for the model, it scores 54.01 %.

Reproducibility

All main runs used Python 3.12 with the package versions fixed in `requirements-lock.txt` (among them tabpfn 6.4.1 with the TabPFN-2.5 weights, torch 2.10.0 with CUDA 12.8, river 0.23.0

and scikit-learn 1.3.2) on one laptop with an NVIDIA RTX 2060 Max-Q GPU (6 GB). The public repository (https://github.com/wenboxia/tabular-fm-temporal-shifts) contains the code, the unit tests, the pre-registration with its amendments, the result reports of rounds 2 to 4 with all supplementary tables and figures, and the per-seed results (`results/round{2,3,4}_per_seed.csv`). The scripts `scripts/reproduce_round{1,2,3,4}.sh` rerun each round; with `SMOKE=1` they run a short check on a CPU in a few minutes. Rounds 2 to 4 use seeds 0 to 9 and round 1 seed 42. The raw result files are too large for the repository and are kept by the author.

Checks. No context contained a future row or exceeded its budget. Replaying all 360 dual-memory runs used in rounds 3 and 4 (20 of them reused from round 2) from the labels alone reproduced the memory fingerprint at every checkpoint, and the GPU runs reproduced the CPU pilot on 99.3–99.6 % of its predictions. With batches of 10 instead of 50 rows (Insects (abrupt), three seeds), the RET gain of the 1000-row regime-balanced dual memory over the 200-row sliding window (the pre-registered baseline, one fifth of the budget) was nearly unchanged ($+12.29$ against $+12.21$ pp).

C PRE-REGISTRATION AND HYPOTHESES

The pre-registration and its amendments are in the public repository (`results/phase56_prereg.md`); each version was committed before the runs it governs, and later changes were added as dated amendments with their reasons.

Original registration (before round 2). It fixed the question, the Insects (abrupt) stream, the holdout sets, the protocol of Section 3.2, the metrics, fourteen policies, eight hypotheses and a go/no-go criterion for a pilot run on a CPU. The proposed method was a *regime archive*: a class-balanced sample of 25 rows per class from each finished regime, added to a 200-row window only when a query resembled that regime more than the window.

Amendment 1 (after the pilot). The archive method failed the go criterion: its ADAPT was 2.71 pp below the 200-row window (limit -2 pp), although its RET was 8.93 pp higher. Queries were routed to old archives for 13–72 % of the current regime's rows, and predictions on those rows were worse than with the window alone. The pilot also showed the long-term part of the class-balanced dual memory drifting towards the latest regimes. The amendment therefore made the regime-balanced dual memory the proposed method, with the hypotheses unchanged except that H7 became a non-inferiority test against the class-balanced dual memory. The new method passed the same go criterion ($+12.6$ pp RET, $+0.93$ pp ADAPT for the 1000-row memory against the 200-row sliding window (the pre-registered baseline, one fifth of the budget)) before any main run.

Amendment 2 (before the main runs). It placed H2 in the Holm family with H1 and H3, required all ten seeds for a formal verdict, and stated the ADWIN settings and the reported quantities.

Amendment 3 (round 3). Written before any full run of round 3, it added the three further streams with the selection criteria of Section 3.4, the holdout rule, the three arms of the gradual stream, the comparison of the three policies at 200 and 1000 rows, and the analysis: a Holm family of the eight RET comparisons between the regime-balanced dual memory and the class-balanced dual memory, the -0.5 pp margin for ADAPT, and no success thresholds. Pipeline checks and a few CPU runs made before the GPU runs, among them retention on Gas for some policies, are listed in a note to the amendment; nothing registered

Table 5: Pre-registered hypotheses of round 2 (Insects (abrupt), 1000-row memories, ten seeds) and their outcome. "Ours" is the regime-balanced dual memory. Differences are in percentage points; 10/10 means positive in all ten seeds.

	Hypothesis	Compared with	Result	Verdict
H1	Ours retains at least 5 more, significantly.	200-row window	$+12.19$ RET, 10/10	holds
H2	Ours adapts no worse: one-sided 95 % lower bound above -0.5, also with single-class runs masked.	200-row window	$+1.16$ ADAPT (bound $+1.08$); masked $+1.75$ ($+1.67$)	holds
H3	Ours retains at least 2 more, significantly.	Class-balanced dual memory	$+1.42$ RET, 10/10, significant	fails
H4	In the 500 rows after a change, ours is at least 5 better after the 2nd and 4th change point and within ± 2 after the 1st and 5th.	200-row window	-6.68, -8.07; -4.21, $+0.68$	fails
H5	No window size is at least as good as ours in both ADAPT and RET.	Windows of 100–2000 rows	none	holds
H6	Ours retains at least 2 more.	The 66 most recent rows of each class	$+0.81$ RET	fails
H7	Ours adapts at most 0.5 worse.	Class-balanced dual memory	$+0.19$ ADAPT	holds
H8	The boundary-free variant keeps at least half of the H1 gain and passes H2.	200-row window	$+11.37$ RET; bounds $+1.11$, $+1.89$	holds

was changed because of them. The separability measure of Section 5.2 was defined after those Gas runs and is exploratory.

Note on the short-term share (round 4). Written after the results of round 3 and before any run of round 4, it registered the shares 0.65 and 0.85 at 200 rows as an exploratory analysis without tests or thresholds.

Hypotheses of round 2. Table 5 gives the hypotheses as amended and their outcome. H1 to H3 are primary: one-sided paired t-tests over the ten seeds, Holm-corrected, $\alpha = 0.05$, with the sign required in at least 9 of 10 seeds. For H2, the one-sided test is of the ADAPT difference plus 0.5 pp, taking the larger p of the raw and masked versions, and at least 9 seeds must show a difference above -0.5 pp. H1, H2, H4 and H8 compare the 1000-row regime-balanced dual memory with the 200-row sliding window (the pre-registered baseline, one fifth of the budget); the same-budget comparisons are in Section 4.3. The originally registered archive method, judged by the original H1 to H3, had a RET 10.41 pp above the 200-row window but an ADAPT 3.24 pp below it (H2 fails) and a RET 0.37 pp below the class-balanced dual memory (H3 fails).

D SUPPLEMENTARY RESULTS

This appendix shows what the long-term part holds on the three streams not shown in Figure 7 (Figure 12), all sixteen policies of round 2 (Figure 13), how well rows of one regime predict another (Figure 14) and the exploratory analysis of the short-term share (Figure 15, Section 4.4). Further tables and figures, including the 1000-row versions and the full analysis of round 4, are in the result reports of the public repository (`results/phase56_round{2,3,4}.md` and `results/round{3,4}_analysis.md`).

Besides the three policies of the main text, round 2 compared the following (Figure 13; mean ADAPT/ RET in %):

- sliding windows of 100, 300, 400, 600, 1500 and 2000 rows, from 73.70 / 33.81 (100 rows) to 75.51 / 42.11 (2000 rows), with the highest ADAPT at 600 rows (77.14);
- a class-balanced dual memory of 400 rows: 76.11 / 43.82;
- the boundary-free regime-balanced dual memory (ADWIN): 76.88 / 46.45;

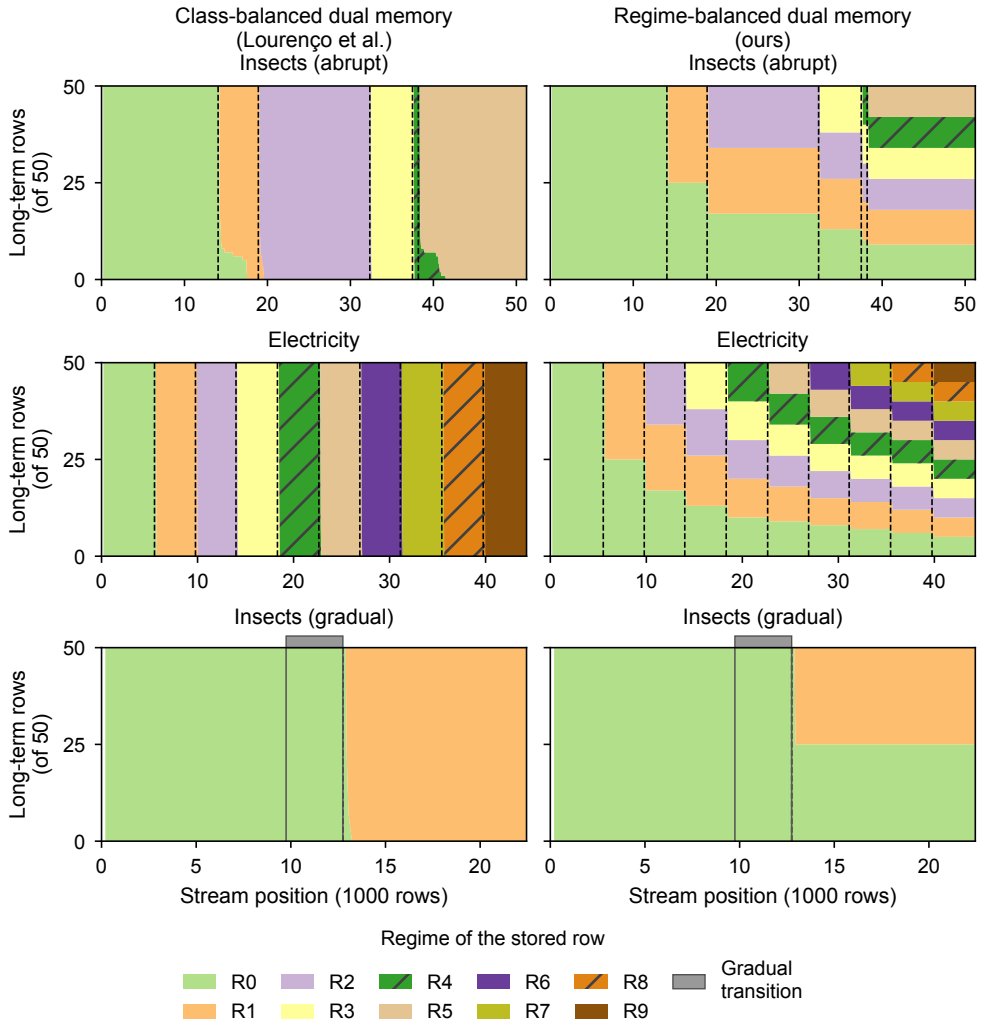


Figure 12: What the 50-row long-term part holds over time on the other three streams (200-row context, one run, seed 0). Colours: regime of each stored row; dashed lines: regime ends; grey bar (Insects (gradual)): the gradual transition, which belongs to R0. The class-balanced dual memory (Lourenço et al. [35]) (left) keeps mainly the newest regime and at the end holds no older rows. The regime-balanced dual memory (ours) (right) shares the long-term part evenly among all regimes seen so far. On Insects (abrupt), the class-balanced dual memory keeps a few old rows of some classes for a while after R0 and R4 end. On Insects (gradual) it replaces R0 within about 450 rows of the documented change point (about 2,900 rows with a 1000-row memory).

- a class-balanced FIFO memory holding the 66 most recent rows of each class (400 rows): 67.97 / 46.47;
- the regime archive with all regimes: the 200-row window plus the archives of all finished regimes, 72.52 / 52.45, the highest retention but 4.3 pp less adaptation than the regime-balanced dual memory;

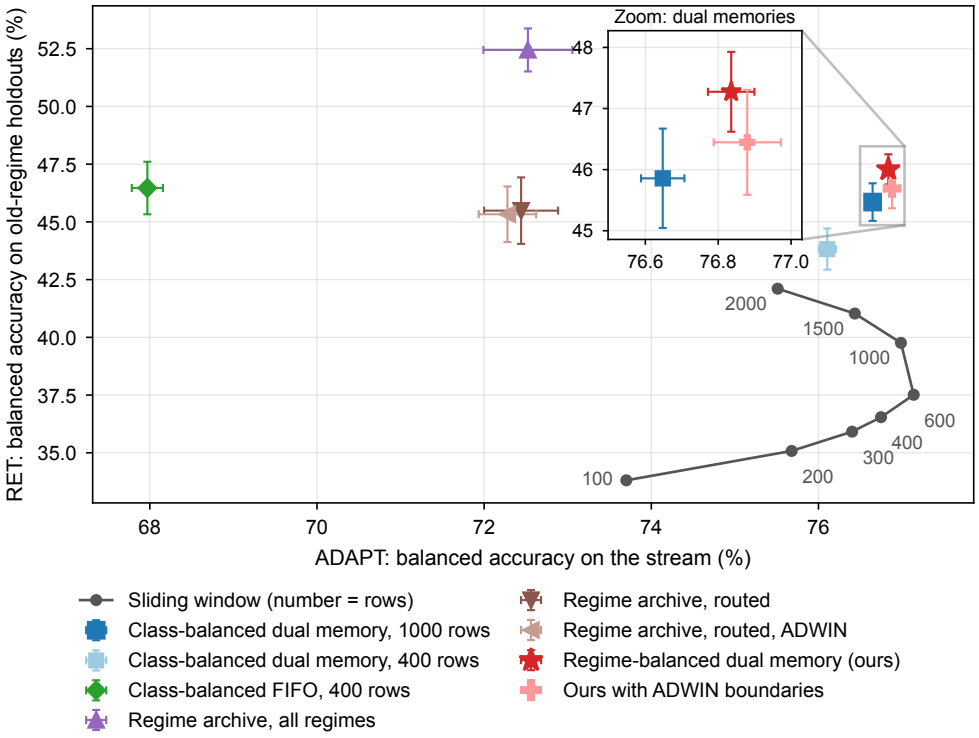


Figure 13: Adaptation and retention of all 16 second-round policies on Insects (abrupt) (mean $\pm$ SD, ten seeds; SD of the sliding-window points ≤ 0.65, not drawn). The grey line joins sliding windows of 100–2000 rows. Compare the 1000-row memories with the 1000-row window; the 200-row window was the pre-registered baseline (one fifth of the budget). The regime archives are the originally registered method (Appendix C). No policy is at least as good as the regime-balanced dual memory in both metrics. Inset: the three 1000-row dual memories.

- the routed regime archive: the 200-row window plus the archive of the finished regime a query resembles most, with documented or ADWIN boundaries, 72.44 / 45.49 and 72.28 / 45.33.

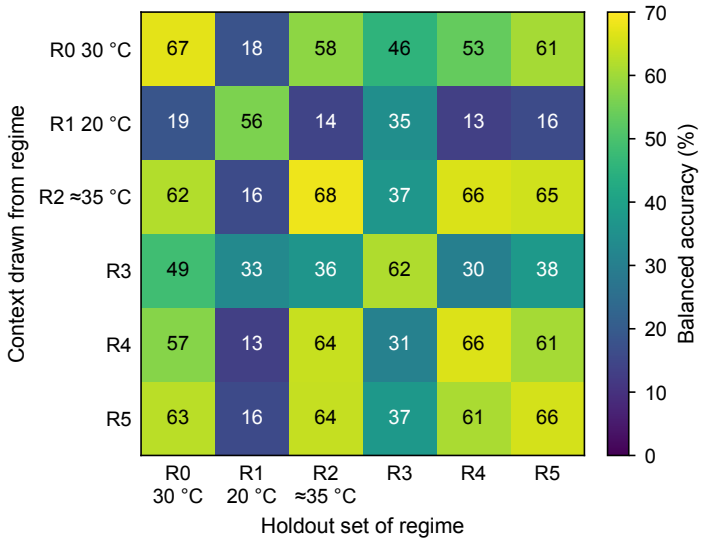


Figure 14: How well a context from one regime predicts another (Insects (abrupt); balanced accuracy in %, mean over ten seeds). Each context holds 198 class-balanced rows drawn from one regime (rows); each column is the holdout set of one regime. The diagonal lies at 56–68 %; R1 is almost unrelated to the other regimes (13–35 % in its row and column), whereas R0, R2, R4 and R5 predict each other well (53–66 %). Forgetting R1 therefore cannot be made up by rows of other regimes.

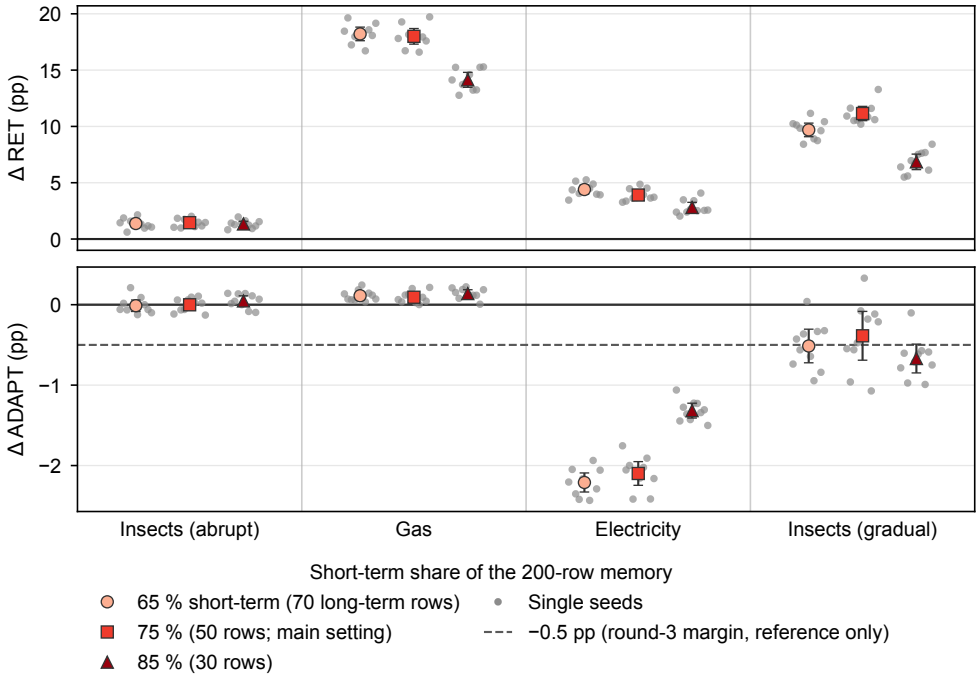


Figure 15: Exploratory analysis: regime-balanced dual memory minus class-balanced dual memory (Lourenço et al. [35]) when the short-term part takes 65, 75 or 85 % of a 200-row memory (75 % is the main setting). Markers: mean over ten seeds with 95 % t intervals; grey dots: single seeds. Top: the RET gain stays positive for every stream, share and seed. Bottom: the cost in ADAPT on Electricity stays beyond the -0.5 pp reference line at every share.

TRITA xxxx:yyy
Stockholm, Sweden 2026

www.kth.se